

GamutX

Virtual Environments

PYTHON SCRIPTING

Introduction to Virtual Environments

- What Is an Environment?
- What is a Virtual Environment?

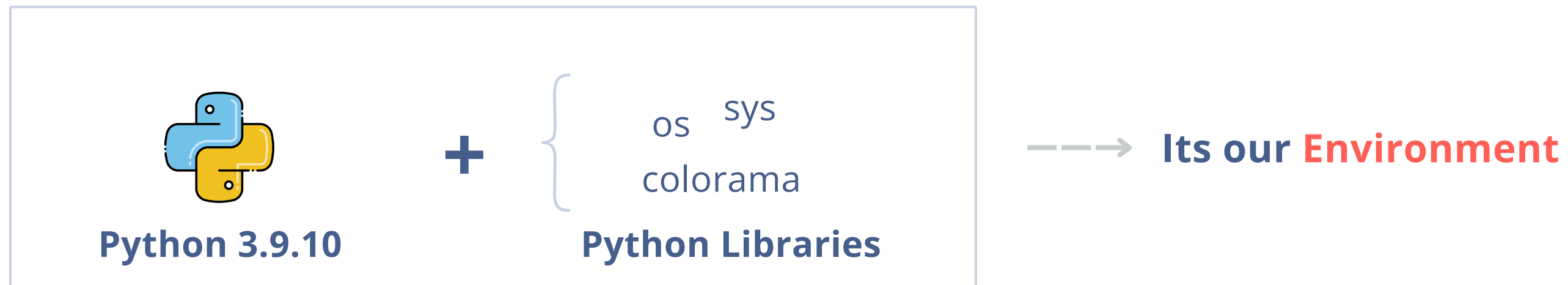
What Is an Environment?

Think of an **environment** like a **sandbox** where your **code** **lives** and **runs**. It includes:

- The **Python version** you're using.
- All **installed libraries** and their specific versions.

If you remember when we started this course,

- We **installed Python** and then added a **few libraries** that we needed. **That setup**



What is a Virtual Environment?

- A **virtual environment** is a **self-contained folder** that holds:
 - A **Python interpreter**
 - A **set of libraries** installed just for one project

It keeps your project **isolated** from the **system-wide Python** and other projects you're working on.

Problem Scenario:

Setup:

- You're developing **two** tools:

Tool Name	Python Version	Required Library	Version
• Tool A	→ 3.7	→ numpy	→ 1.21.0
• Tool B	→ 3.11	→ numpy	→ 1.26.0

You install **numpy 1.26.0** globally to make **Tool B** work.

When you run Tool A:

you get:

```
AttributeError: module 'numpy' has no attribute 'matrixmultiply'
```

Why?

- Tool A was built and tested with numpy 1.21.0
- numpy 1.26.0 removed or changed functionality that Tool A relies on
- Because **pip install numpy modified the global environment, it affected both tool.**

This is **common in studios** where **scripts** are passed between **TDs, artists,** and **render nodes.**

✓ **Solution: Use Virtual Environments**

- Each tool should **use its own virtual environment.**
- This keeps Python versions and **packages isolated**, so **changes in one tool won't affect another.**

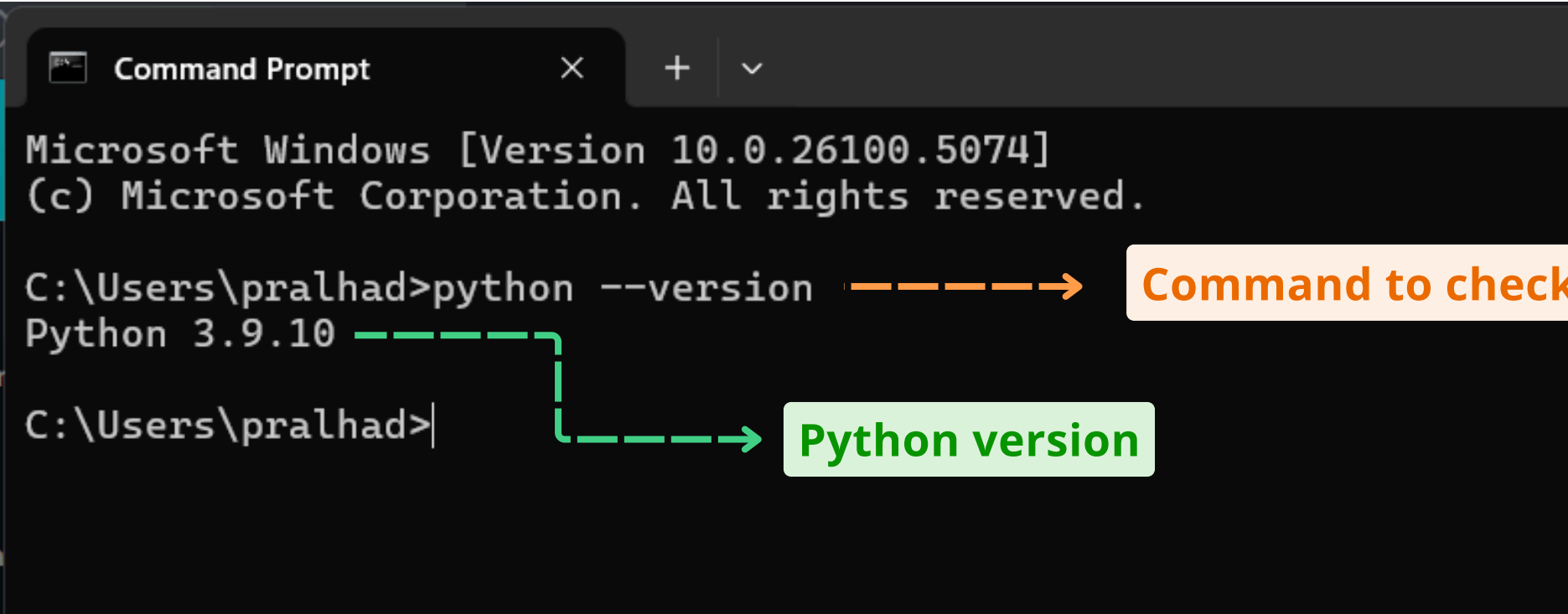
Creating Virtual Environments

- Check Your Python Version
- Creating a Virtual Environment
- Handling Execution Policy Errors
- What's Inside the .venv Folder?
- Why Use .venv as the Folder Name?
- Why This Matters

Check Your Python Version

Before creating a virtual environment, let's first check that Python is installed correctly.

- Open your **terminal** or **command prompt** and type:



```
Command Prompt
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>python --version
Python 3.9.10

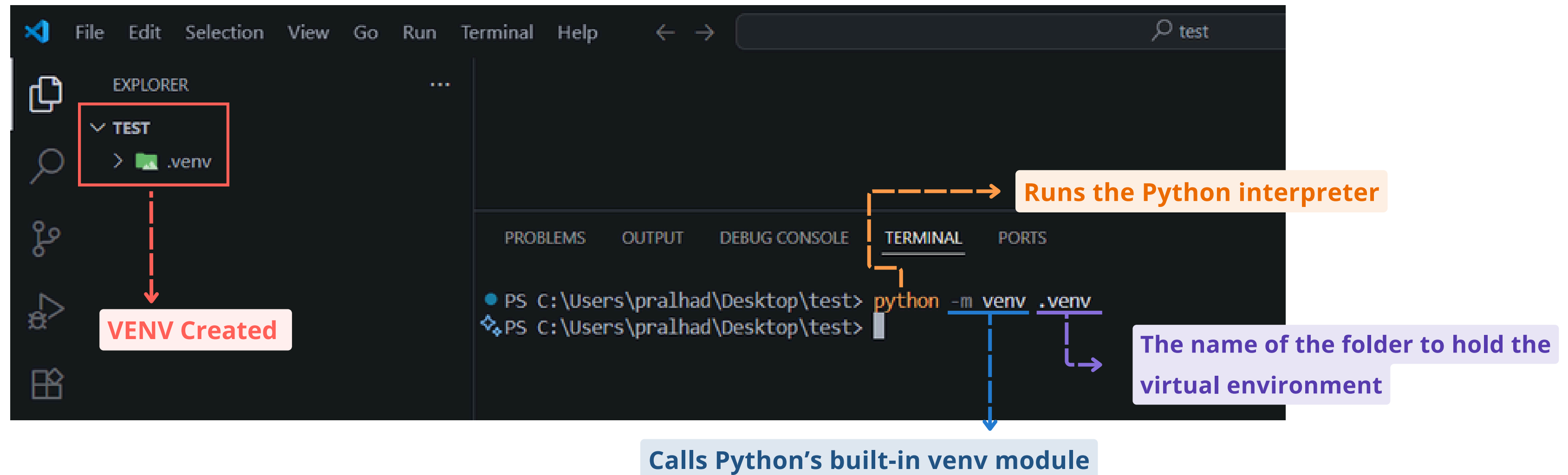
C:\Users\pralhad>
```

Command to check python version

Python version

Creating a Virtual Environment

- Decide where your project will live. For example, a **folder** on your **desktop**:
- Open Your Folder on **VS Code** and in **Terminal Type**



- **.venv** is a common convention, especially in professional projects.
- You can name it differently, but **.venv** is often auto-detected by tools and ignored by version control.

Activate the Virtual Environment

- Before using the environment, you need to activate it.
- This tells your **terminal** to use the **Python interpreter** and **packages** from **.venv**.

The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the following commands and output:

```
PS C:\Users\pralhad\Desktop\test> .venv\Scripts\activate
❖(.venv) PS C:\Users\pralhad\Desktop\test>
```

Annotations with arrows point to specific parts of the terminal output:

- A purple arrow points from the text "Includes Python, pip, and activation scripts" to the command `.venv\Scripts\activate`.
- A red dashed arrow points from the text "script that activates the environment" to the same command `.venv\Scripts\activate`.
- A blue dashed arrow points from the text "Indicate that VENV is Activated" to the output `❖(.venv)`.

Handling Execution Policy Errors

When we **activate** a **virtual environment** for the **first time** on **Windows**, PowerShell may **block** the **activation script** and show an error like:

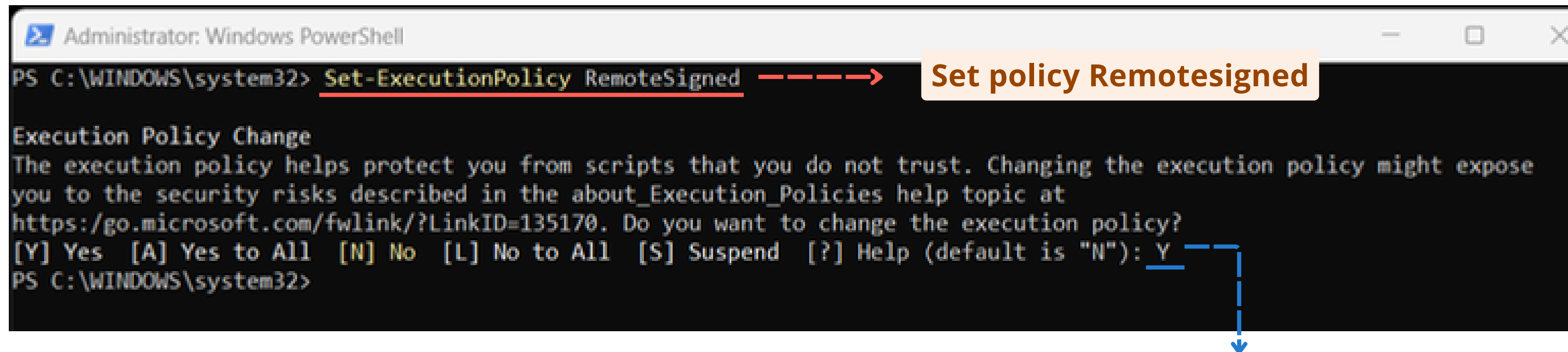
```
.venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled on this system.
```

What “Execution Policy” Means

- PowerShell’s execution policy **controls whether scripts are allowed to run or not**
- It helps prevent malicious scripts, but it can also block local, harmless scripts like your venv activator.

Setting the Execution Policy

- Open PowerShell as Administrator.
- Run the following command to change the execution policy:



```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned -----> Set policy Remotesigned

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the execution policy might expose
you to the security risks described in the about_Execution_Policies help topic at
https://go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): Y
PS C:\WINDOWS\system32>
```

Confirm the change by typing **Y** when prompted.

- Lets local scripts (like **.venv\Scripts\Activate**) run.
- Keeps stricter defaults for the rest of the system.

Common Policy Levels

Restricted :

- **Blocks** all scripts from running. **Default setting** for Windows.

Remote Signed :

- **Allows local scripts** to run; remote scripts must be signed by a **trusted publisher**.

Unrestricted :

- **Allows all scripts** to run, including **unsigned ones** from the **internet**. Poses a **security risk**.

Bypass :

- Allows all scripts to run without **any warnings**. Disables **protections**.

AllSigned :

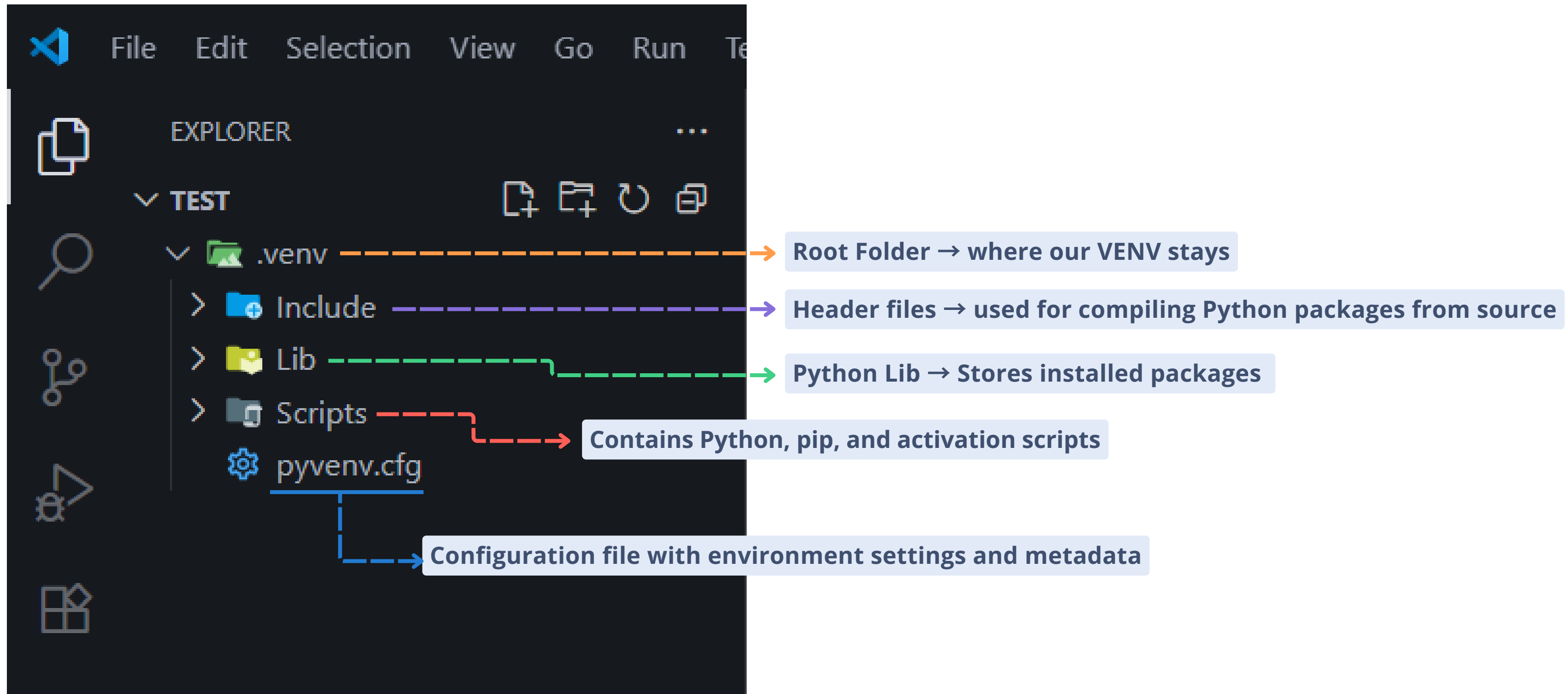
- Only allows **scripts** signed by a **trusted publisher**, both **local** and **remote**.

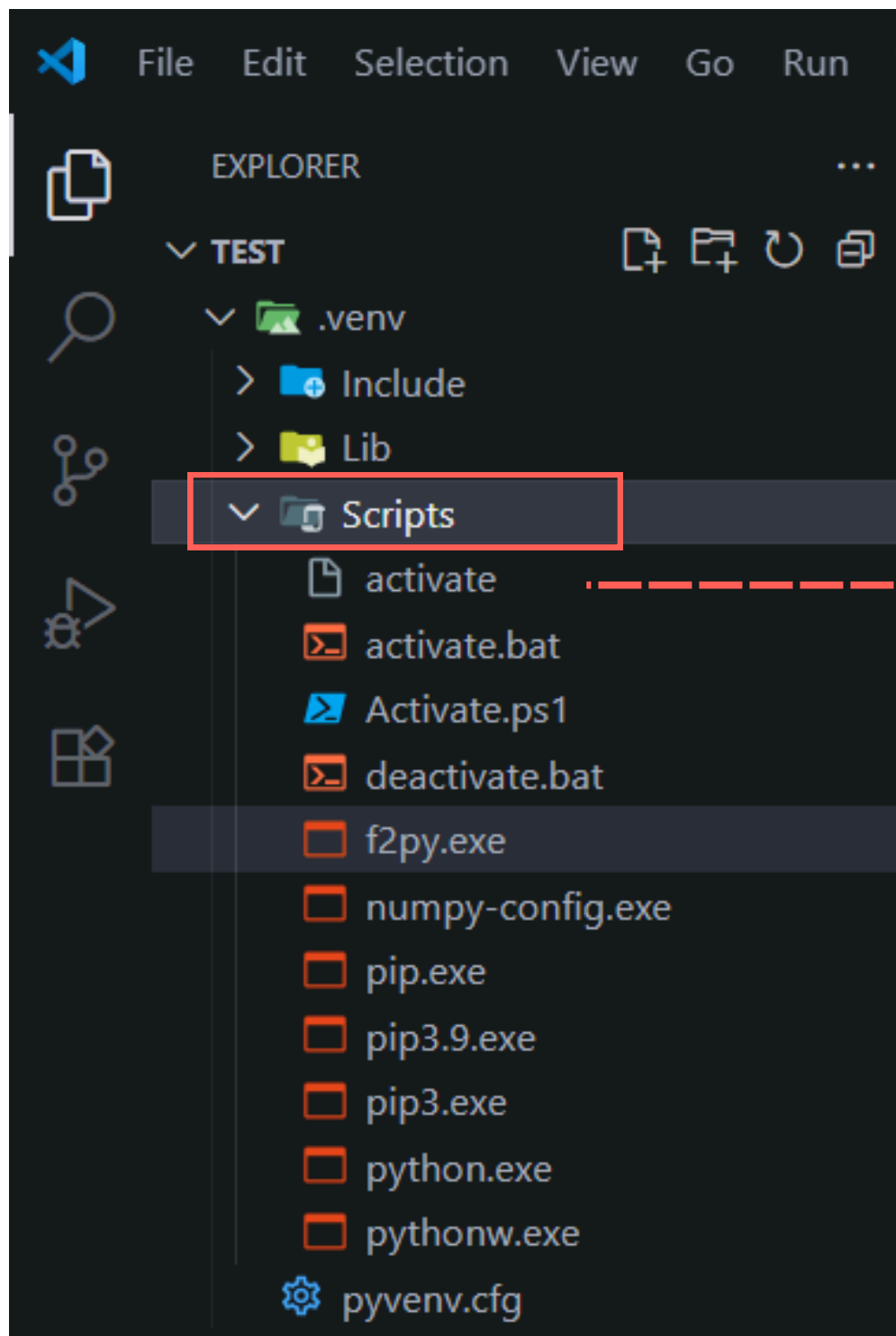
Undefined :

- No execution policy is set. **Default** behavior if **unsure**.

What's Inside the .venv Folder?

After running the command, a new directory named venv will be created in your project folder.





bin/ (on macOS/Linux) or Scripts/ (on Windows)

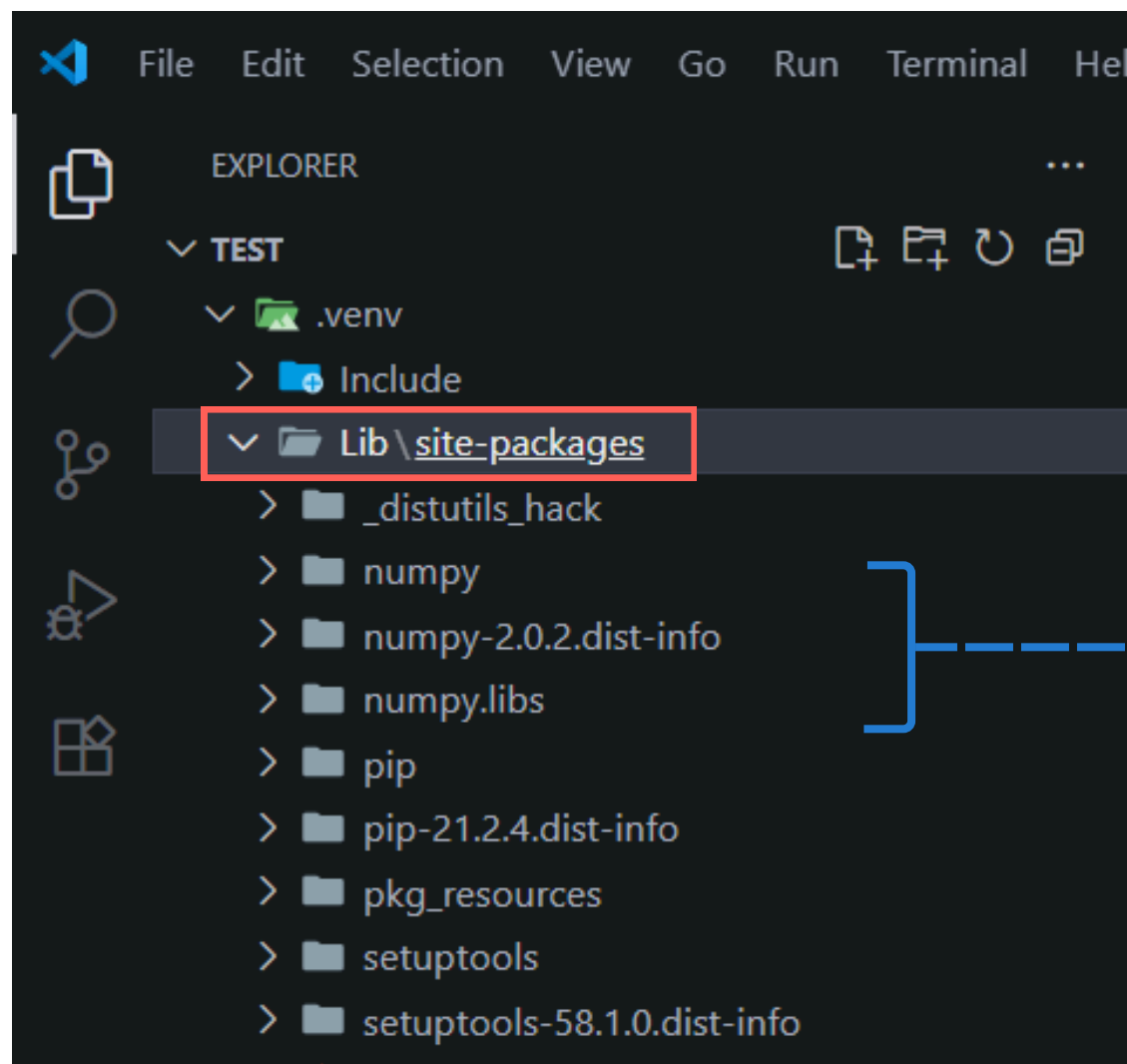
This is where the **executable files live**. Think of it as the control room.

Includes:

- A **copy of the Python interpreter** — specific to this **virtual environment**
- The **pip** tool for installing **Python packages**
- **Activation scripts** to enable the environment in your terminal

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pralhad\Desktop\test> .venv\Scripts\activate
(.venv) PS C:\Users\pralhad\Desktop\test> 
```



lib/ (or Lib/ on Windows)

This is the **library folder**—basically where all your installed packages live.

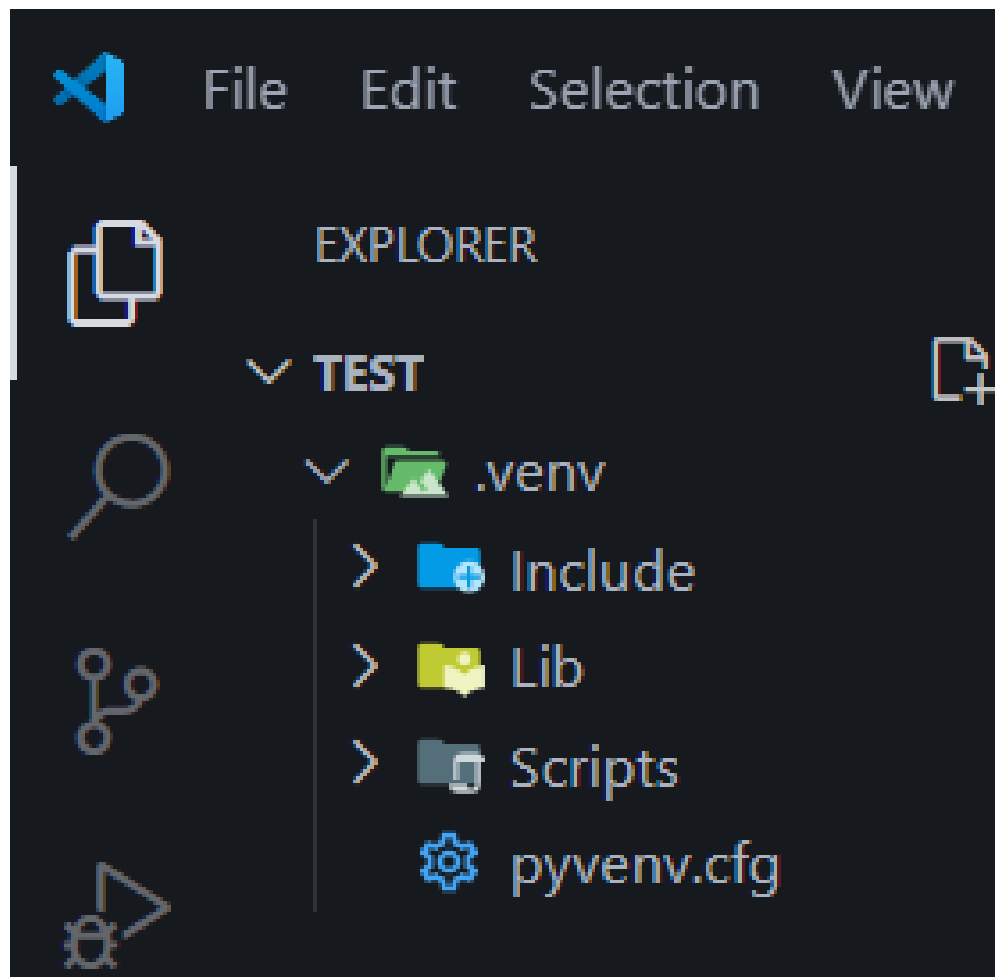
Let's say we do → `pip install numpy`

Inside this site-packages/ directory, you'll find:

- The actual **numpy/ package folder**
- **Metadata folders** like **numpy-<version>.dist-info/**

Why this matters:

- Each project gets its own package space
- No conflicts with other tools or environments
- Cleaner collaboration in studio pipelines



include/

Contains **C header files** used when building **Python packages** from **source**.

When Is It Used?

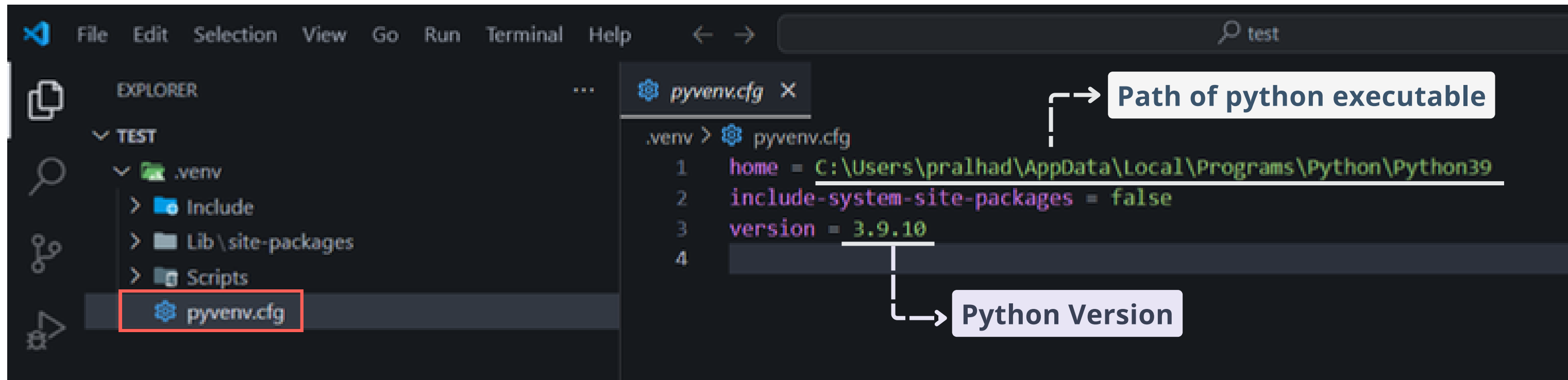
We typically **don't interact with this folder** unless We're:

- Compiling packages written in **C or C++**
- Installing a package that **includes native extensions**
 - e.g., NumPy, OpenImageIO, PyOpenGL
- Writing or building **Python bindings for C++ plugins or APIs**

Real World Use Case

- If a TD is building a custom plugin in C++ that wraps Python, this folder provides the necessary headers during compilation.

pyvenv.cfg



This is a small configuration file that stores **metadata** about your **virtual environment**.

It usually contains:

- The Python version used
- The path to the base Python installation
- A few other environment settings

Why Use .venv as the Folder Name?

1. Standardization

- **.venv** is a **widely recognized convention** in the Python community.
- Keeps your project structure:
 - **Predictable** for collaborators
 - **Cleaner** for version control and tooling

In a team or studio setting, consistent folder naming saves time and avoids confusion.

2. Integration with Tools

- Most Python tools and editors (like **VS Code**, **PyCharm**, **CI systems**, etc.) are designed to work well with **.venv**.
- **Automatically ignored by version control**



```
.gitignore U X
.gitignore
1 .venv/
2 venv/
3 env/
4 ENV/
5
```

Why This Matters

Creating a virtual environment gives your project its **own isolated Python setup**, separate from your **system Python**.

- **No version clashes**

Different projects can use **different versions** of libraries (e.g., **PySide2** in one, **PySide6** in another).

- **No interference between projects**

Dependencies are **local to the project** — what you install in one project won't affect others.

- **Easier cleanup**

To **remove** a virtual environment, just delete the **.venv** folder. No system changes.

- **Better portability**

Share your project with others using a **requirements.txt** file. They can recreate the same ENV reliably.

Working with Virtual Environment

- Selecting the Virtual Environment Interpreter in VSCode
- Global Python Environment vs Virtual Environment
- Where Does a Package Get Installed in a Virtual ENV?
- Viewing Installed Packages in a Virtual Environment?
- Comparing Global vs Virtual Environment
- Deactivating the Virtual Environment
- Key Features of Virtual Environments

Selecting the Virtual Environment Interpreter in VSCode

Create a Python File

- Create a file named **example.py** in your project folder.
- Add a simple **print statement**:



The screenshot shows the VS Code interface. The editor has a file named `example.py` with the following content:

```
1 print("This is running without a virtual environment!")
2
```

The terminal at the bottom shows the command to run the file using the global Python interpreter:

```
PS C:\Users\pralhad\Desktop\test> & C:\Users\pralhad\AppData\Local\Programs\Python\Python311\python.exe c:/Users/pralhad/Desktop/test/example.py
This is running without a virtual environment!
PS C:\Users\pralhad\Desktop\test>
```

A callout box with the text "this is global python path" points to the Python interpreter path in the terminal command. The status bar at the bottom right of the editor shows the current Python interpreter as "Python 3.9.10".



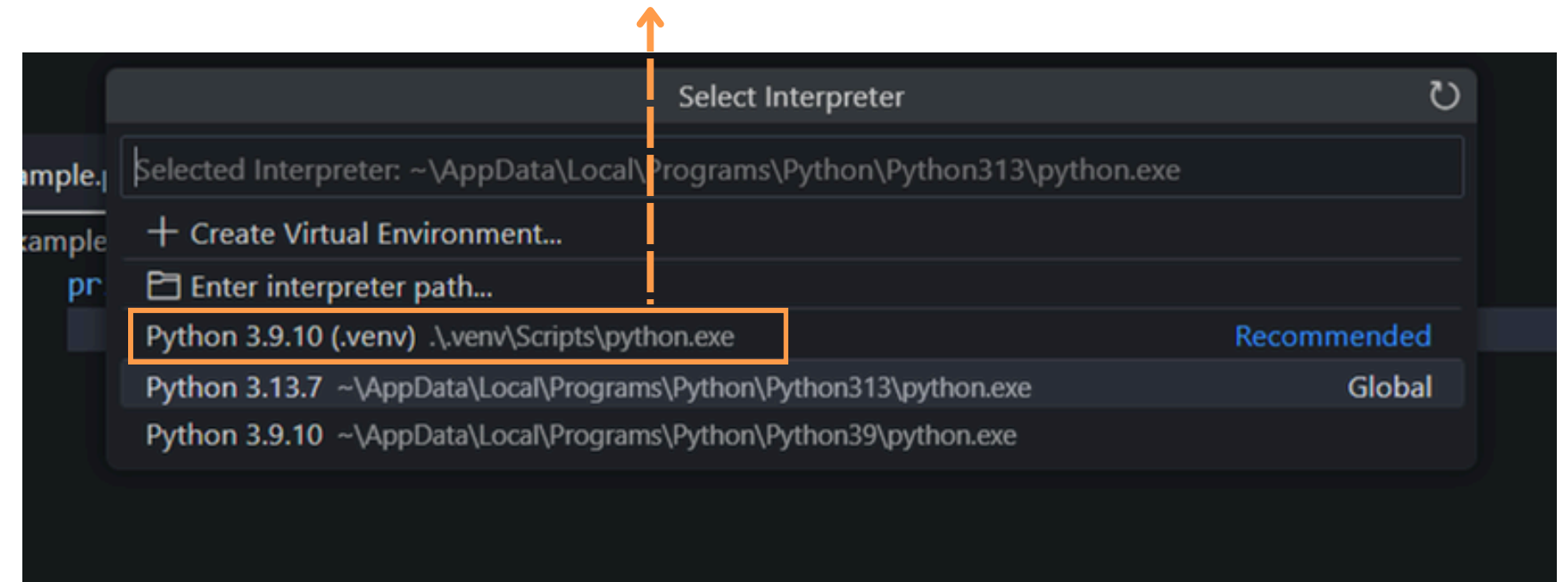
⚠ At this point, our project is still using the system-wide Python, not an isolated environment.

Selecting the .venv Interpreter



click to select python interpreter

Select the VENV python path



Running with the Virtual Environment

- Now when you run **python example.py**, VS Code uses the **.venv** interpreter.
- The integrated terminal will **automatically activate** the **virtual environment** for you.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pralhad\Desktop\New folder> & "C:/Users/pralhad/Desktop/New folder/.venv/Scripts/Activate.ps1"
```

```
• (.venv) PS C:\Users\pralhad\Desktop\New folder> python .\example.py
```

```
This is running without a virtual environment!
```

```
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> █
```

→ It indicate that we are inside Virtual ENV

Global Python Environment vs Virtual Environment

Imagine this situation

- We've installed the **colorama** library **globally** on **our machine**.
- But we haven't **installed** it inside our **virtual environment** yet.

Test in the Global Python Environment

Open your project and create a file called **example.py**:

```
example.py x
example.py
1  import colorama
2
3  print("Colorama is imported successfully!")
4
5
```

Import colorama

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [] []

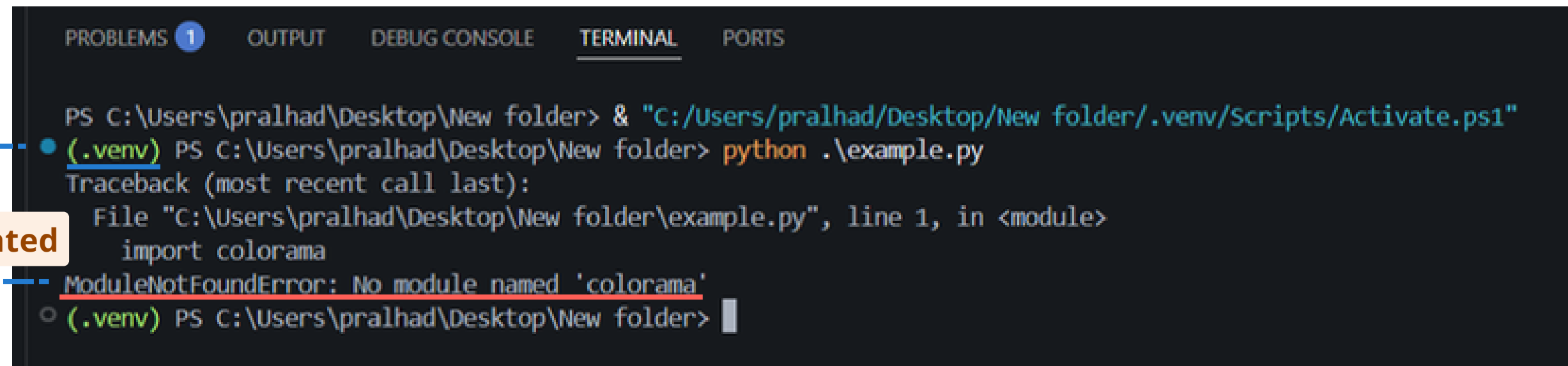
```
PS C:\Users\pralhad\Desktop\New folder> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39/python.exe "c:/Users/pralhad/Desktop/New folder/example.py"
● Colorama is imported successfully!
○ PS C:\Users\pralhad\Desktop\New folder> |
```

Using Global Python

Because **colorama** exists in our **global Python** installation, the script runs just fine.

Now Switch to the Virtual Environment

Switch over to the interpreter inside your **.venv** — either by activating it in the **terminal** or **selecting** it in **VS Code**.



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pralhad\Desktop\New folder> & "C:/Users/pralhad/Desktop/New folder/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\pralhad\Desktop\New folder> python .\example.py
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\New folder\example.py", line 1, in <module>
    import colorama
ModuleNotFoundError: No module named 'colorama'
(.venv) PS C:\Users\pralhad\Desktop\New folder>
```

Virtual ENV Activated

get ModuleNotFoundError

→ Because **colorama** only installed in **globally** not inside our **VENV**

Why This Happens

Environment	access global packages?	Has colorama?	Result
• Global	→  Yes	→  Yes	→ Script runs
• Virtual (.venv)	→  No	→  No	→ Script throws error

Solution

First, make sure you're **inside your project folder.** and **Virtual ENV activated**

Install the Missing Library

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\pralhad\Desktop\New folder> & "C:/Users/pralhad/Desktop/New folder/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\pralhad\Desktop\New folder> pip install colorama
Collecting colorama
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: colorama
Successfully installed colorama-0.4.6
WARNING: You are using pip version 21.2.4; however, version 25.2 is available.
You should consider upgrading via the 'C:\Users\pralhad\Desktop\New folder\.venv\Scripts\python.exe -m pip inst
(.venv) PS C:\Users\pralhad\Desktop\New folder> 
```

installing colorama using pip

- This installs **colorama** only inside this **virtual environment**.

Where Does a Package Get Installed in a Virtual ENV?

- When we install a **library** with **pip** inside a **virtual environment**, it doesn't go into **global Python**.
- Instead, it's placed in the **site-packages** folder of your **.venv**.

On Windows

```
.\venv\Lib\site-packages
```

- A folder named **colorama** (the actual library code)
- A **colorama-x.x.dist-info** folder with metadata about the installation

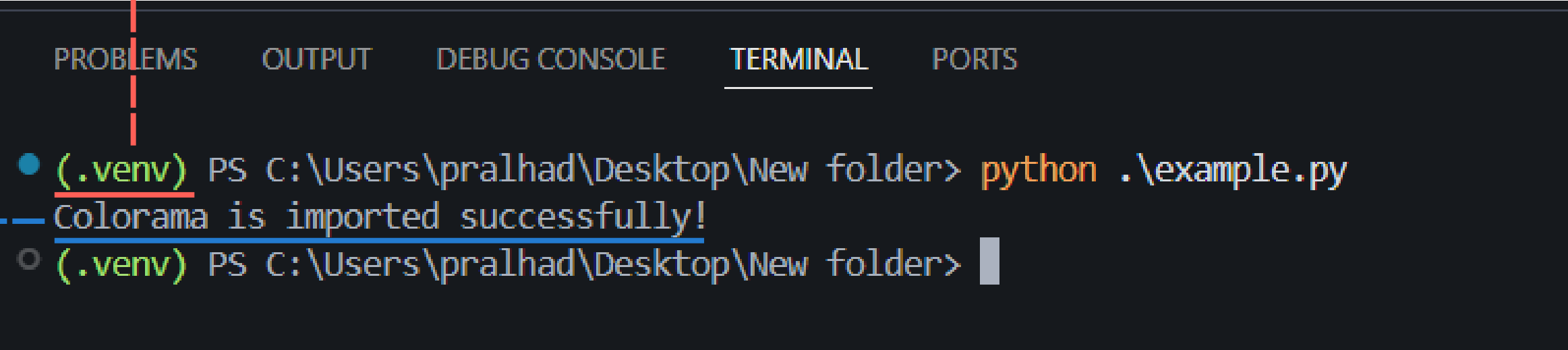
On macOS/Linux

```
./venv/lib/python3.x/site-packages
```

- **colorama/** with the library files
- **colorama-x.x.dist-info/** with version info and metadata

Run Your Script Again

Now try running the script once more:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal output shows a successful execution of a Python script. A red dashed arrow points from a callout box to the `(.venv)` prefix in the command prompt. A blue dashed arrow points from another callout box to the output line.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

• (.venv) PS C:\Users\pralhad\Desktop\New folder> python .\example.py
Colorama is imported successfully!
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> █

✓ Problem solved!

- The library is now available inside your virtual environment.
- our project is self-contained — no global dependencies required.
- Anyone else can recreate the same setup just by installing into their `.venv`.

Viewing Installed Packages in a Virtual Environment?

When working in Python, it's important to know where your packages live — global vs project-specific.

Using pip

With your **virtual environment activated**, run:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• (.venv) PS C:\Users\pralhad\Desktop\New folder> pip list
Package      Version
-----
colorama      0.4.6
pip           21.2.4
setuptools    58.1.0
WARNING: You are using pip version 21.2.4; however, version 25.2 is available
You should consider upgrading via the 'C:\Users\pralhad\Desktop\New folder\.'
○ (.venv) PS C:\Users\pralhad\Desktop\New folder>
```

shows all installed libraries

list of libraires installed inside Virtual ENV

Comparing Global vs Virtual Environment

Without activation (global environment):

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\pralhad\Desktop\New folder> pip list
Package Version
-----
altgraph 0.17.4
annotated-types 0.7.0
anyio 4.10.0
beautifulsoup4 4.13.5
certifi 2025.8.3
charset-normalizer 3.4.3
colorama 0.4.6
contourpy 1.3.0
cycler 0.12.1
decorator 5.2.1
exceptiongroup 1.3.0
fonttools 4.59.2
```

You'll usually see a **long list** of packages, including:

- Libraries installed by your **IDE** (e.g., VS Code, PyCharm)
- System tools
- Packages from other projects

With activation (virtual environment):

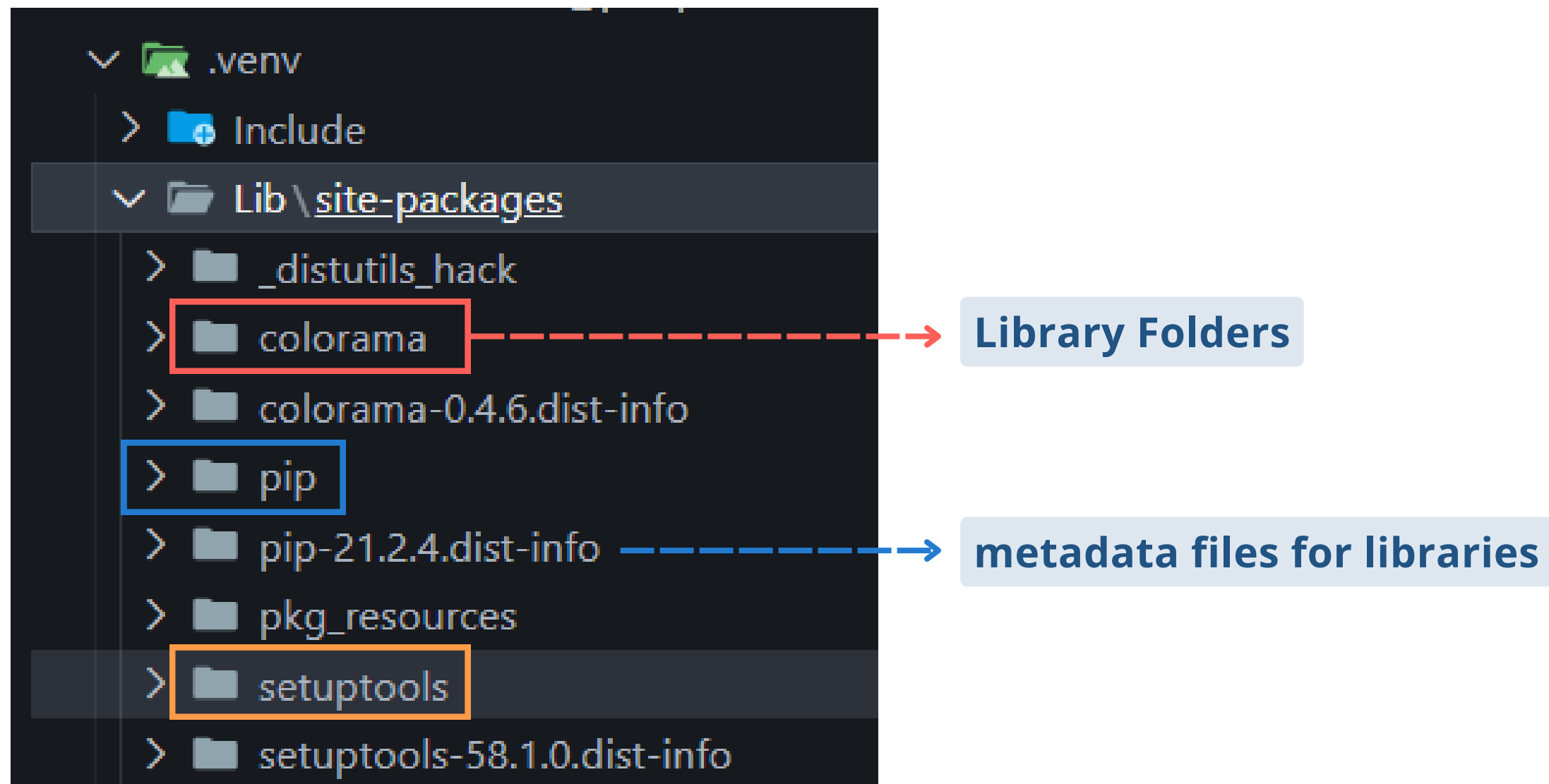
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• (.venv) PS C:\Users\pralhad\Desktop\New folder> pip list
Package Version
-----
colorama 0.4.6
pip 21.2.4
setuptools 58.1.0
WARNING: You are using pip version 21.2.4; however, version 25.2 is available
You should consider upgrading via the 'C:\Users\pralhad\Desktop\New folder\venv\Scripts\python.exe --upgrade pip'
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> 
```

- This is your **isolated project environment**
- it contains only what you've explicitly installed.

Checking Manually in the Folder

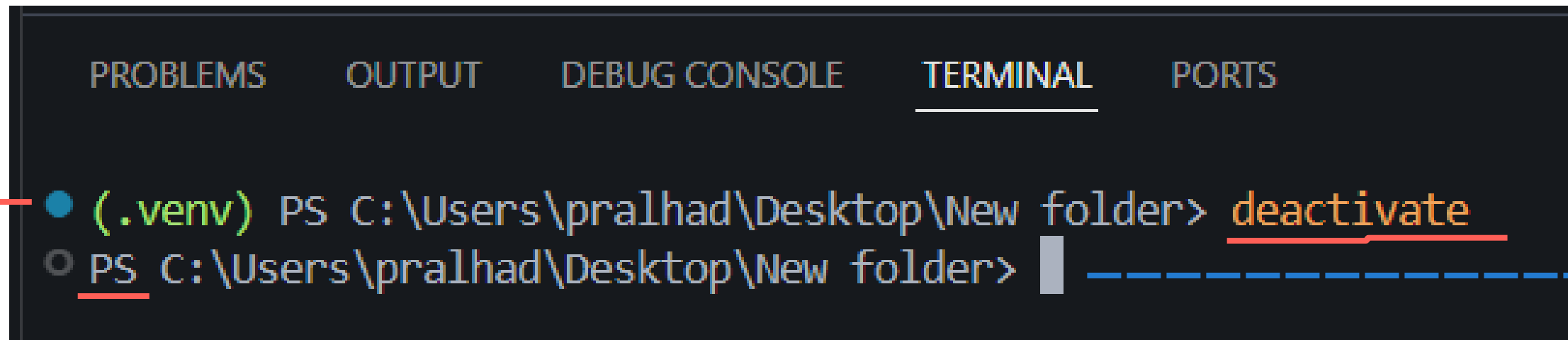
You can also look directly in the **site-packages** folder:

- **Windows** → `.\venv\Lib\site-packages\`



Deactivating the Virtual Environment

- When we finish working on a project, it's good practice to **deactivate the virtual environment**.
- This switches our terminal back to using the **global Python installation**.



The screenshot shows a VS Code terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The terminal output shows a prompt with a blue dot and a green (.venv) prefix: `(.venv) PS C:\Users\pralhad\Desktop\New folder> deactivate`. The word `deactivate` is underlined in red. The next line shows the prompt has changed to `PS C:\Users\pralhad\Desktop\New folder>`, with a dashed blue arrow pointing from the previous line to this new prompt.

shell returns to its normal state.

The (venv) prefix disappears from your terminal prompt.

Any future **python** or **pip** commands will use the **global interpreter**, not the one in **.venv**.

Note:

- We don't need to **delete** the environment to stop using it **deactivate** simply ends the session.

Key Features of Virtual Environments

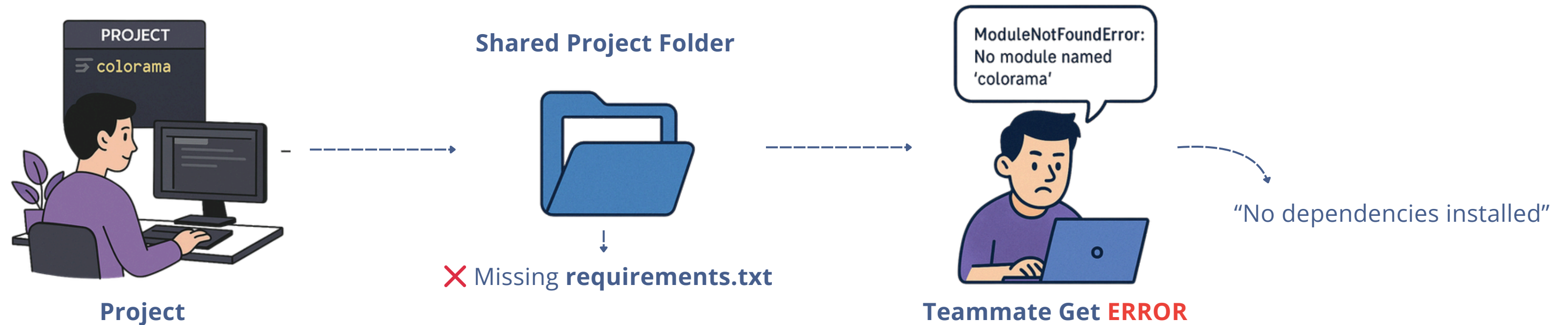
Feature	What It Means	Why It Matters
• Own Python Interpreter	→ Each env uses its own Python version	→ Avoids conflicts with system Python
• Isolated Library Directory	→ Only installs packages for one project	→ Keeps projects independent
• No Global Interference	→ Changes here don't affect other environments	→ Safe to test and update dependencies

Working with requirements.txt

- What is requirements.txt?
- Creating a requirements.txt File (Manually)
- EXAMPLE - of requirement.txt
- Creating a requirements.txt File (Automatically)
- Share Project Dependencies

Problem Scenario

We built a **Python tool** using third-party packages like **colorama**, installed in a **virtual environment**.



! Why?

- The packages were **installed locally** inside **our .venv**
- But our teammate **doesn't have them**
- We **didn't share which packages to install**

----- ✓ Solution ----->

"Use **requirements.txt** to share project's dependencies."

When our Python **project** uses **third-party libraries** like **colorama**, **numpy**, or **requests**,

We need a way to:

-  **Track** which libraries the project depends on
-  **Share** that list with teammates
-  **Reinstall** those **packages** later — **exactly as needed**

That's where **requirements.txt** comes in.

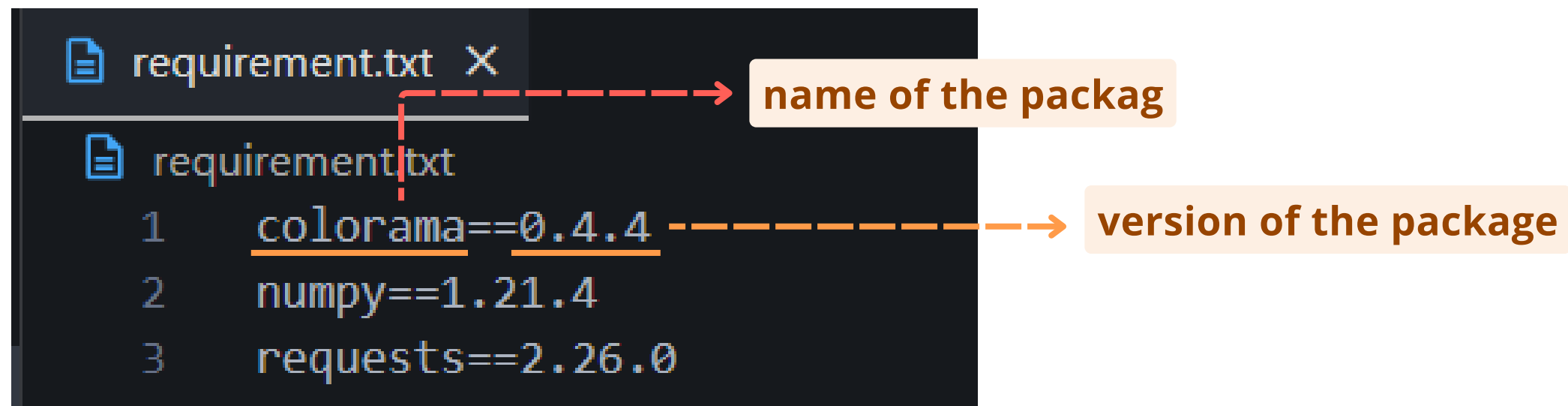


What is requirements.txt?

- It's a **plain text file** that **lists all the Python packages (and versions)** your **project needs**.
- It **does not include**:
 - Python itself
 - System libraries (OS packages)
 - The virtual environment

Sample requirements.txt

A **requirements.txt** file might look like this:



The image shows a code editor window with a file named 'requirement.txt'. The file contains three lines of text: '1 colorama==0.4.4', '2 numpy==1.21.4', and '3 requests==2.26.0'. Two orange callout boxes with arrows point to specific parts of the first line. The first box, labeled 'name of the packag', points to 'colorama'. The second box, labeled 'version of the package', points to '==0.4.4'.

```
requirement.txt X
requirement.txt
1  colorama==0.4.4
2  numpy==1.21.4
3  requests==2.26.0
```

Each line = package name + exact version.

Two Ways to Create **requirements.txt**

Manually

Create the file and list the libraries

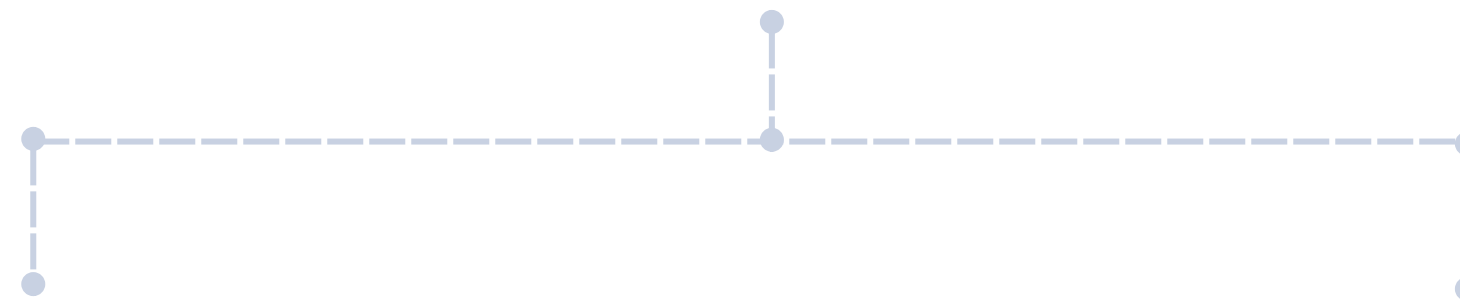
Automatically

Generate the file using a CLI command

Creating a requirements.txt File (Manually)

Creating the file manually gives you full control over which libraries and versions are included.

Identify the Packages Used in Project



Check Python files for import statements

```
example.py X
example.py
1 import colorama
2
3 print("Colorama is imported successfully!")
4
```

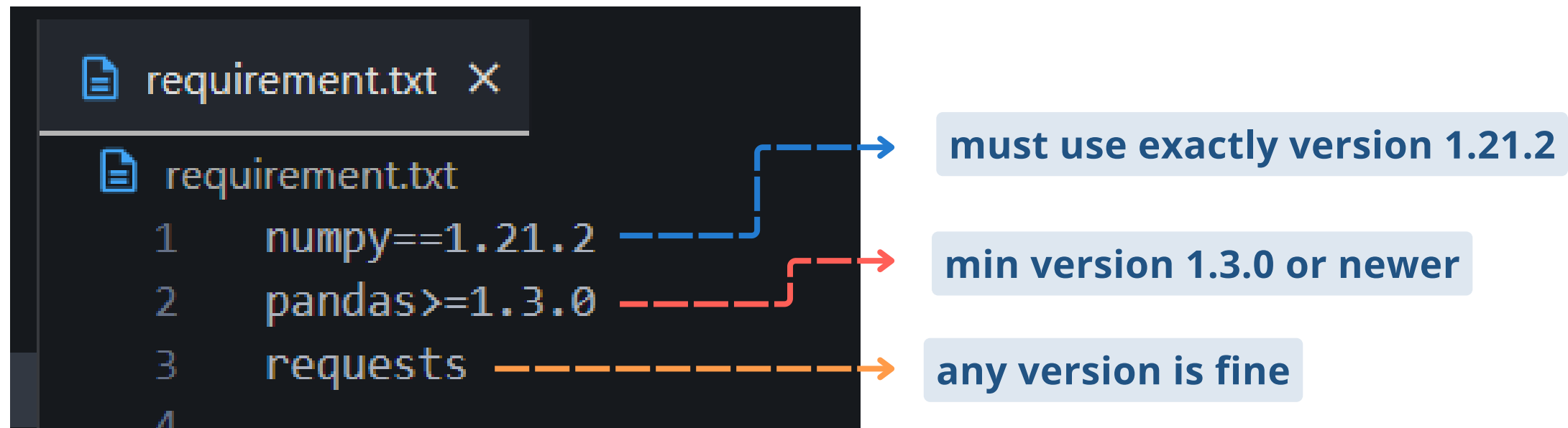
We use colorama

Or list installed packages:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\pralhad\Desktop\New folder> pip list
Package Version
-----
colorama 0.4.6
pip 21.2.4
setuptools 58.1.0
WARNING: You are using pip version 21.2.4; however, version 25.2 is available
You should consider upgrading via the 'C:\Users\pralhad\Desktop\New folder\venv\Scripts\python.exe -m pip install --upgrade pip'
(.venv) PS C:\Users\pralhad\Desktop\New folder>
```

Add Packages to the File

- Create the File with name **requirement.txt**
- Write the **package names** one per line.



We can also **include version numbers** using common operators:

- **==** for an exact version
- **>=** for a minimum version
- **<** or **<=** for upper limits if needed

Install using pip

We can now share it or use it to **install packages**:

-r → read flag

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(.venv) PS C:\Users\pralhad\Desktop\New folder> pip install -r requirement.txt
Collecting numpy==1.21.2
  Downloading numpy-1.21.2-cp39-cp39-win_amd64.whl (14.0 MB)
    |████████████████████| 14.0 MB 3.3 MB/s
Collecting pandas>=1.3.0
  Downloading pandas-2.3.3-cp39-cp39-win_amd64.whl (11.4 MB)
    |████████████████████| 11.4 MB 3.3 MB/s
```

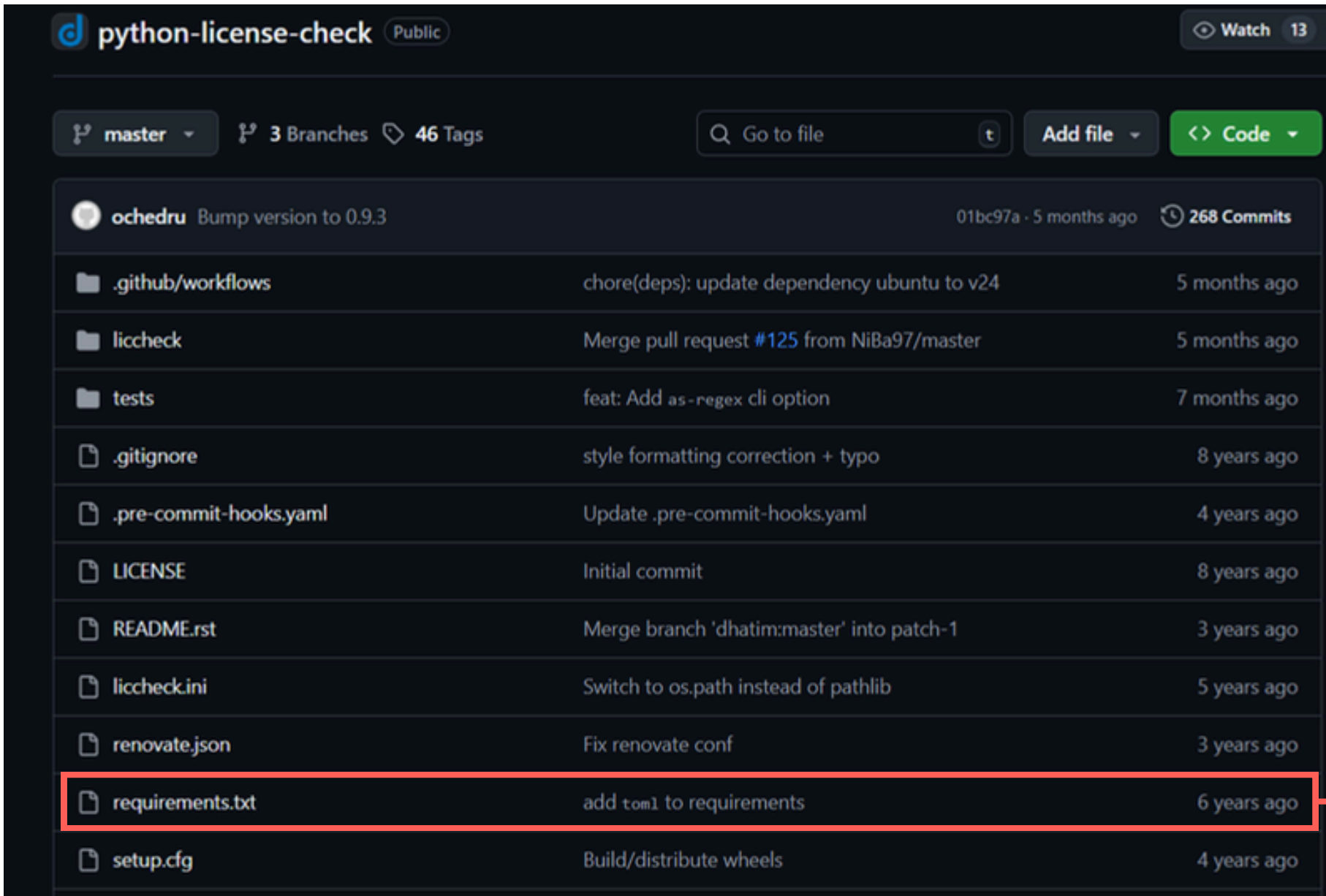
This install libs mention in txt file

When to Use This

- You **know exactly** which libraries your project uses
- You want to **control version ranges** or keep the file minimal

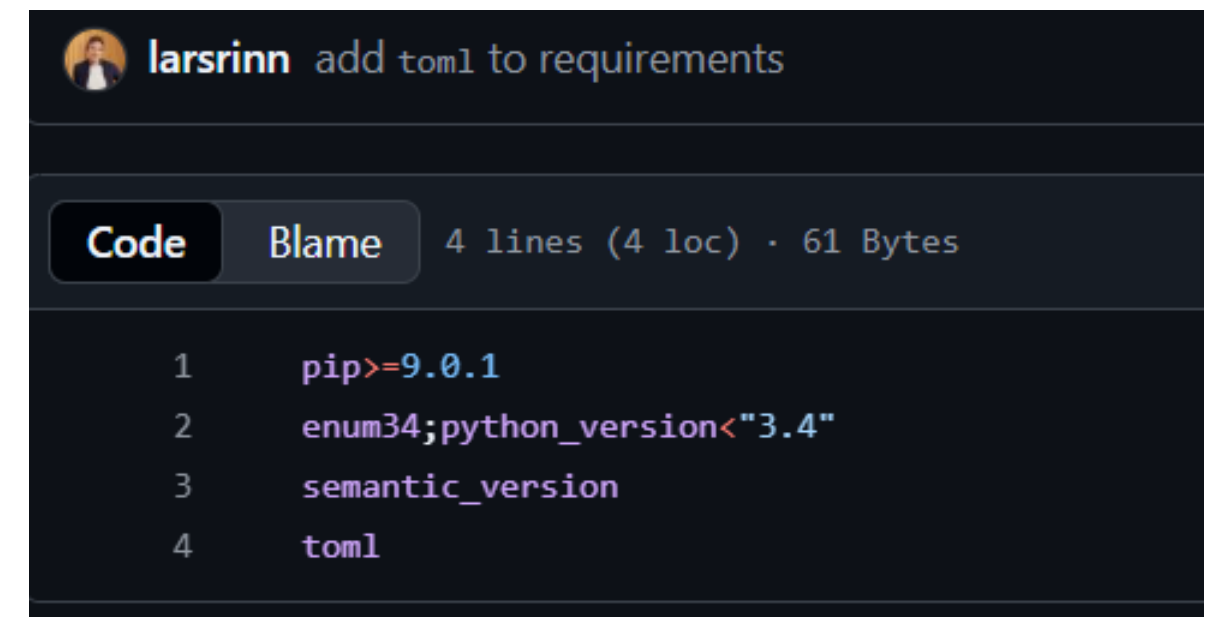
EXAMPLE - of requirement.txt

Most Python projects on GitHub include a requirements.txt file to list **third-party dependencies** needed to **run the project**.



The screenshot shows the GitHub repository 'python-license-check' (Public). The file list includes:

File	Commit Message	Time Ago
.github/workflows	chore(deps): update dependency ubuntu to v24	5 months ago
liccheck	Merge pull request #125 from NiBa97/master	5 months ago
tests	feat: Add as-regex cli option	7 months ago
.gitignore	style formatting correction + typo	8 years ago
.pre-commit-hooks.yaml	Update .pre-commit-hooks.yaml	4 years ago
LICENSE	Initial commit	8 years ago
README.rst	Merge branch 'dhatim:master' into patch-1	3 years ago
liccheck.ini	Switch to os.path instead of pathlib	5 years ago
renovate.json	Fix renovate conf	3 years ago
requirements.txt	add toml to requirements	6 years ago
setup.cfg	Build/distribute wheels	4 years ago



The screenshot shows the commit details for 'requirements.txt' by user 'larsrinn' with the message 'add toml to requirements'. The code content is displayed as follows:

```
1 pip>=9.0.1
2 enum34;python_version<"3.4"
3 semantic_version
4 toml
```

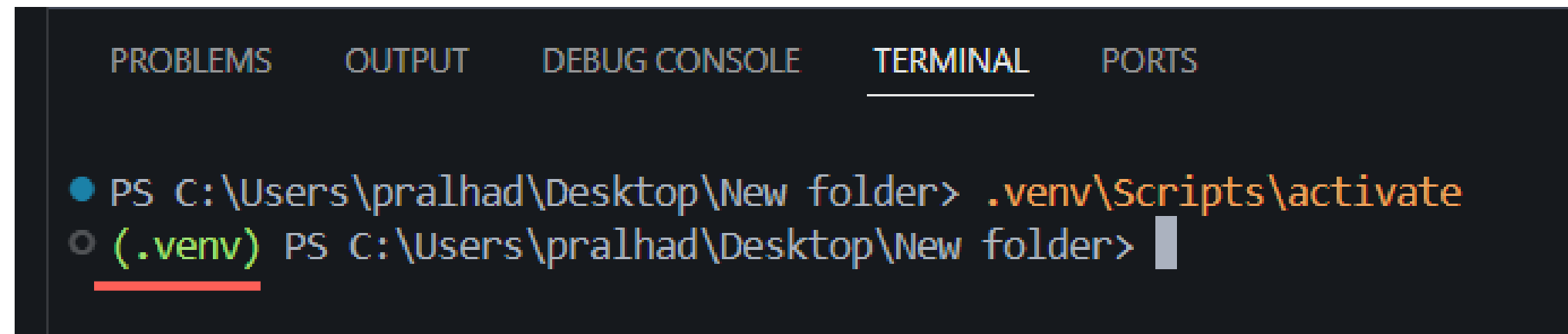
Link → <https://github.com/dhatim/python-license-check>

Creating a requirements.txt File (Automatically)

- Instead of writing package names **manually**, you can **generate** **requirements.txt** using a **single command**.
- This is the fastest and **most reliable way** — especially when you have multiple **libraries installed**.

Activate Virtual Environment

- **Before generating the file**, make sure you're in the **right environment**:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\pralhad\Desktop\New folder> .venv\Scripts\activate
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> |
```

View Installed Libraries

before we generate the **requirements.txt** file, it's useful to see all the **libraries installed** in our **virtual environment**.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• PS C:\Users\pralhad\Desktop\New folder> .venv\Scripts\activate
• (.venv) PS C:\Users\pralhad\Desktop\New folder> pip list
Package            Version
-----
certifi            2025.10.5
charset-normalizer 3.4.3
colorama           0.4.6
idna               3.10
numpy              1.21.2
pandas             2.0.3
pip                21.2.4
python-dateutil    2.9.0.post0
pytz               2025.2
requests           2.32.5
setuptools         58.1.0
six                1.17.0
tzdata             2025.2
urllib3            2.5.0

WARNING: You are using pip version 21.2.4; however, version 25.2 is available.
You should consider upgrading via the 'C:\Users\pralhad\Desktop\New folder\.venv\Scripts\python.exe -m pip install --upgrade pip' command.
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> 
```

- This gives us an overview of the installed libraries,
- but it's not in the format we need for the **requirements.txt** file.

Capture Dependencies

- instead of copying all of these manually, we can use the **pip freeze** command.
- This command will give us the exact format we need for the **requirements.txt** file.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• (.venv) PS C:\Users\pralhad\Desktop\New folder> pip freeze
certifi==2025.10.5
charset-normalizer==3.4.3
colorama==0.4.6
idna==3.10
numpy==1.21.2
pandas==2.0.3
python-dateutil==2.9.0.post0
pytz==2025.2
requests==2.32.5
six==1.17.0
tzdata==2025.2
urllib3==2.5.0
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> 
```

list of libraries installed inside virtual ENV

Generate requirements.txt

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• (.venv) PS C:\Users\pralhad\Desktop\New folder> pip freeze > requirements.txt
○ (.venv) PS C:\Users\pralhad\Desktop\New folder> 
```

outputs every installed package

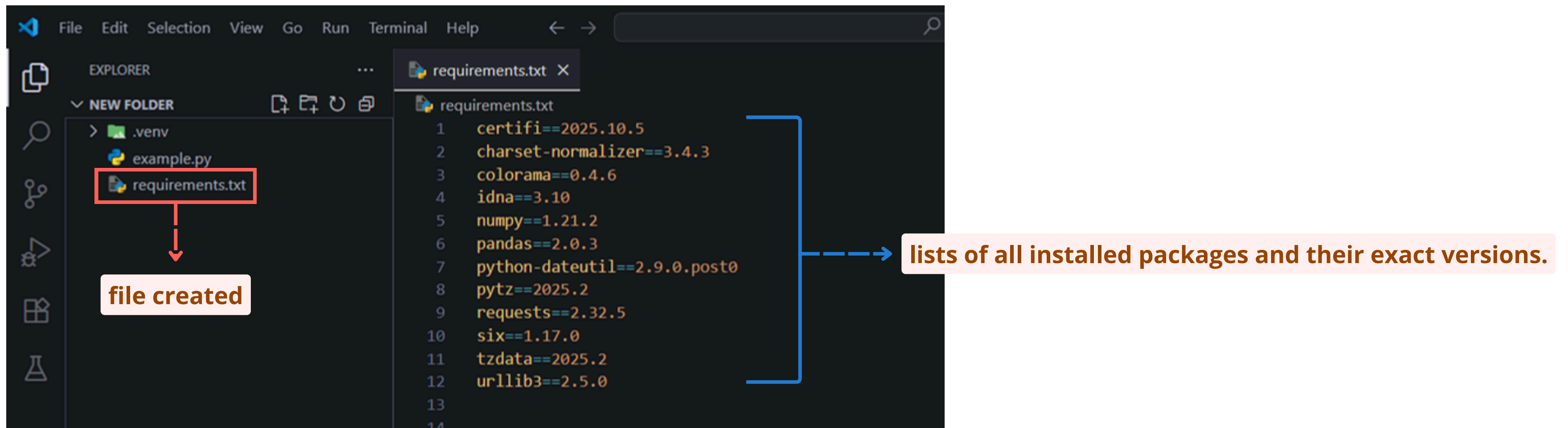
format → package==version

The > sends output into a file named requirements.txt

```
requirements.txt X
requirements.txt
1  certifi==2025.10.5
2  charset-normalizer==3.4.3
3  colorama==0.4.6
4  idna==3.10
5  numpy==1.21.2
6  pandas==2.0.3
7  python-dateutil==2.9.0.post0
8  pytz==2025.2
9  requests==2.32.5
10 six==1.17.0
11 tzdata==2025.2
12 urllib3==2.5.0
```

File is automatically created in CWD

What Happens After You Run It?



Why This Is Useful

- We don't need to list packages manually.
- We won't miss any dependencies.
- Others can install everything with: `pip install -r requirements.txt`

This ensures **everyone runs the same setup, without guessing or troubleshooting missing packages.**

Share Project Dependencies

When you're ready to share your Python project with someone else, it's important to also share the list of libraries your project depends on. That's what the requirements.txt file is for.

Manually Create the requirements.txt File

If you haven't already, create a plain text file named:

```
requirements.txt X
requirements.txt
1  numpy==1.21.2
2  pandas==1.3.3
3  requests==2.26.0
4  flask==2.0.1
```



Place It in Project Folder

Folder Structure



place it in the **main folder** of your project

Share the Project

You can share your project (with the **requirements.txt** file included) in several ways:

- **GitHub** – Upload the project to a **public** or **private repository**.
- **Email** – Send the folder as a **zip file**.
- **Cloud Storage** – Use services like **Google Drive** or **Dropbox**.

What Others Will Do

When someone else receives your project folder, they will:

- Create and activate their **own virtual environment**.
- Install the libraries listed in **requirements.txt** by running:

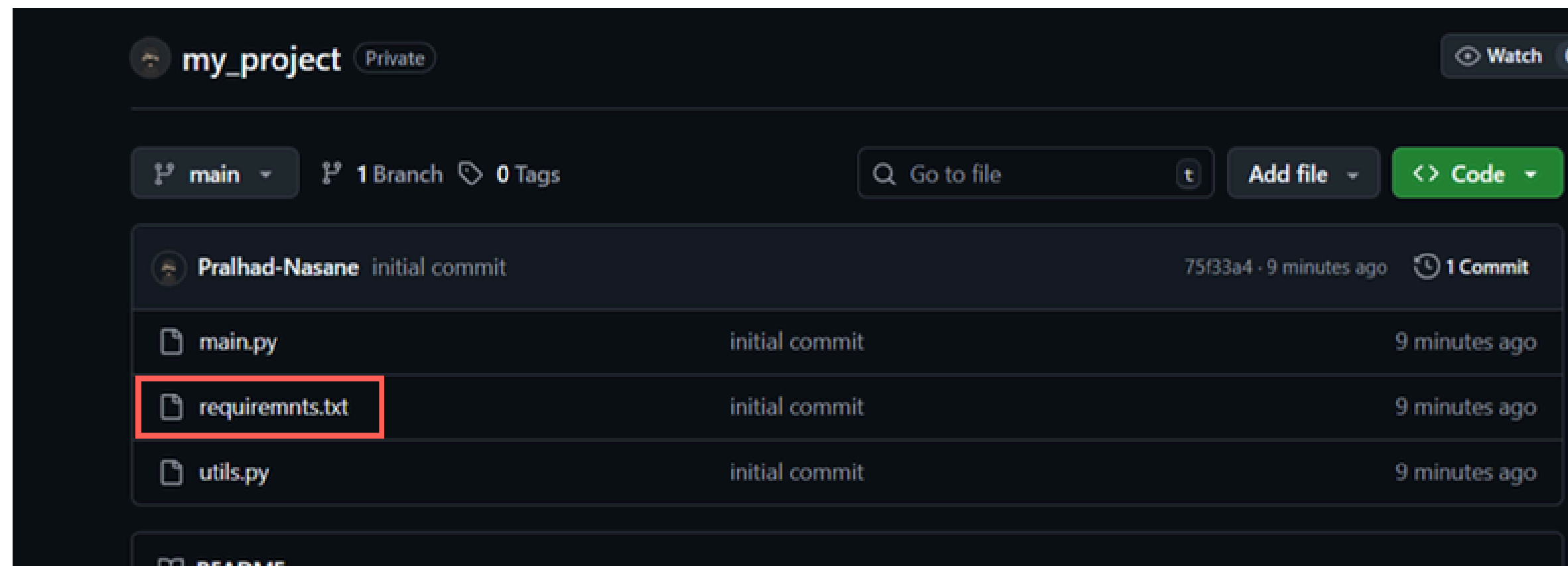
```
pip install -r requirements.txt
```

This will install the **same libraries** you used, ensuring the project runs correctly on **their system**.

Example of a GitHub Repository with requirements.txt

If you are **sharing your project** on **GitHub**, you can include a **requirements.txt** file in your repository, like this:

GitHub Repository Structure:



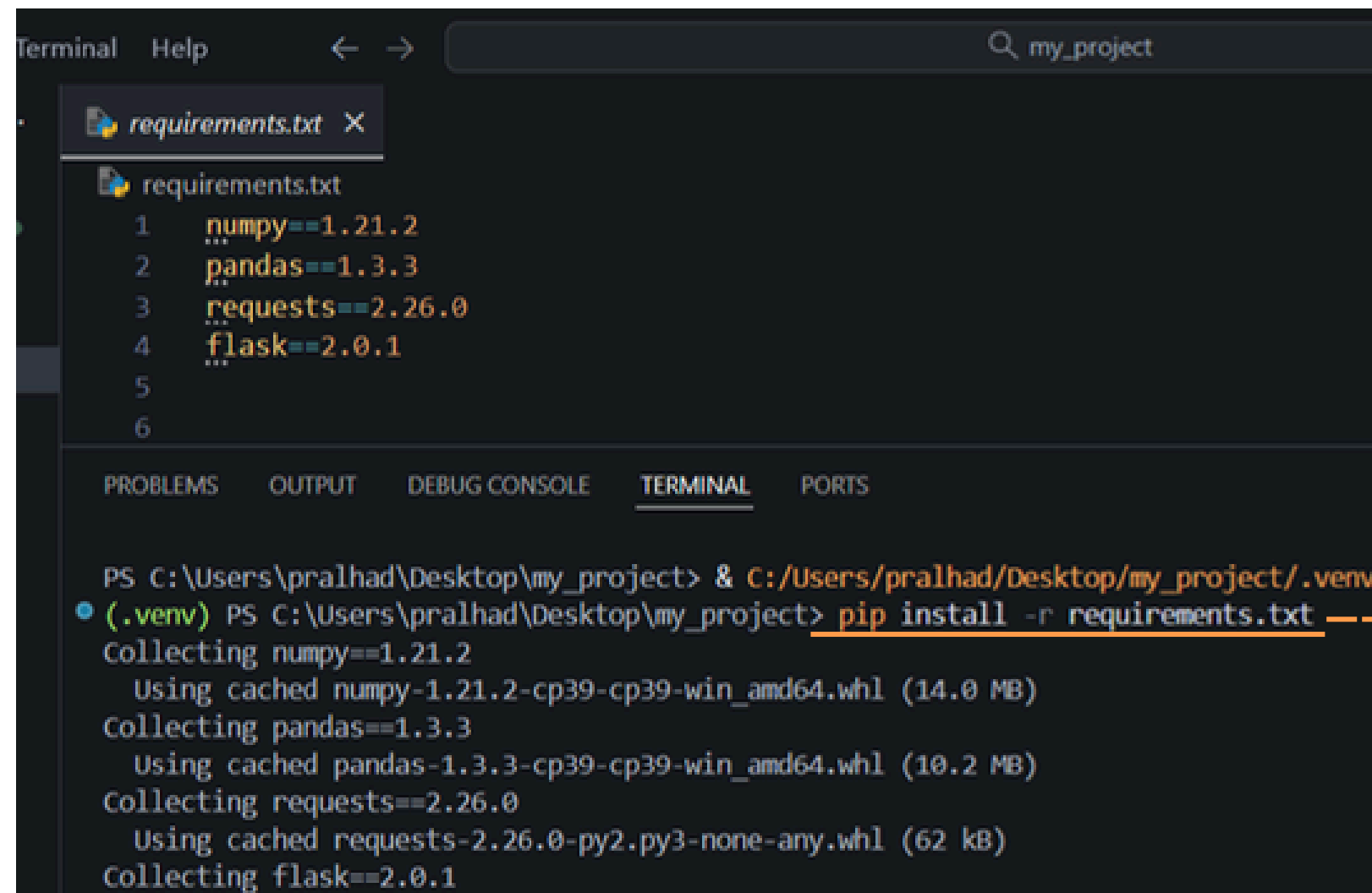
The requirements.txt file will list all the dependencies, so anyone who clones or downloads your project will know exactly which libraries are required to run it.

Problem Scenario



Why It's Important

When someone **clones a project**, they can set up the **required environment** by running:



The screenshot shows a VS Code editor with a file named `requirements.txt` open. The file contains the following content:

```
1 numpy==1.21.2
2 pandas==1.3.3
3 requests==2.26.0
4 flask==2.0.1
5
6
```

Below the editor, the terminal window is active, showing the command `pip install -r requirements.txt` being executed. The output shows the collection of each library and its size:

```
PS C:\Users\pralhad\Desktop\my_project> & C:/Users/pralhad/Desktop/my_project/.venv/
(.venv) PS C:\Users\pralhad\Desktop\my_project> pip install -r requirements.txt
Collecting numpy==1.21.2
  Using cached numpy-1.21.2-cp39-cp39-win_amd64.whl (14.0 MB)
Collecting pandas==1.3.3
  Using cached pandas-1.3.3-cp39-cp39-win_amd64.whl (10.2 MB)
Collecting requests==2.26.0
  Using cached requests-2.26.0-py2.py3-none-any.whl (62 kB)
Collecting flask==2.0.1
```

This installs all the necessary libraries and versions in one step.

Benefits

- **Consistency** → Everyone on the project uses the same versions of each library.
- **Efficiency** → New collaborators can install all dependencies quickly.
- **Portability** → Makes it easier to deploy or move the project across systems.

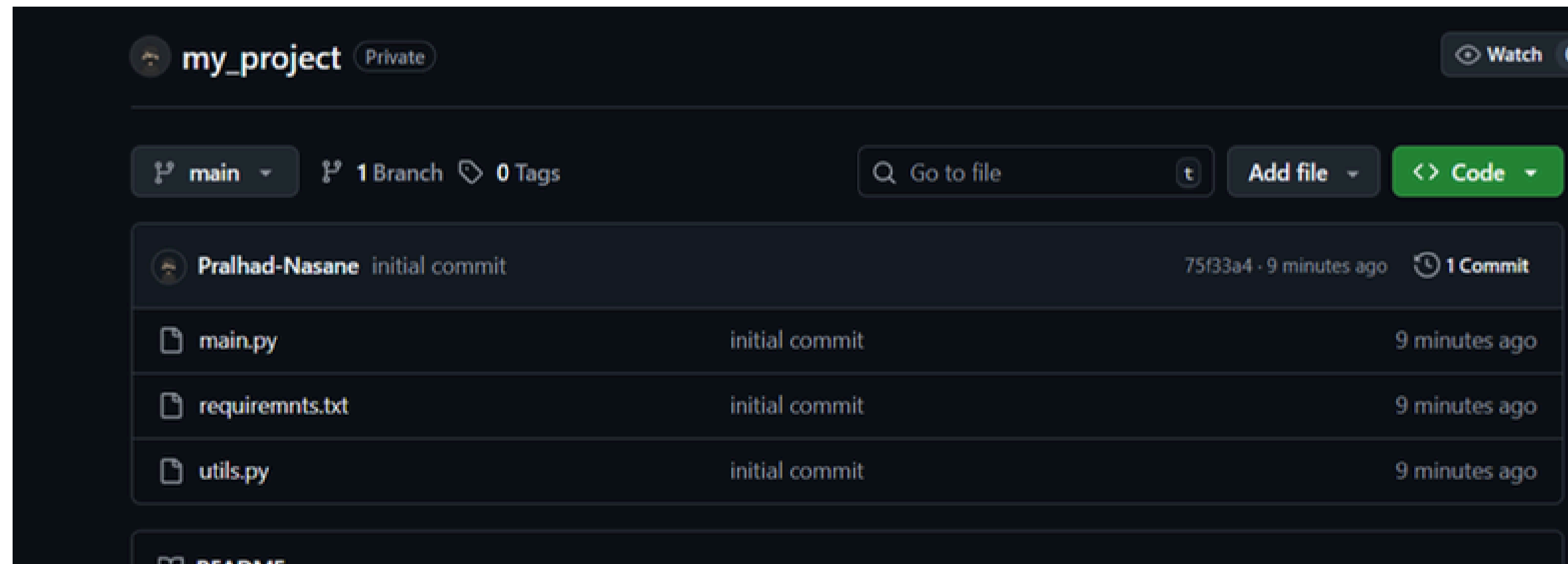
Shared Python Project - Dev. Workflow

- Installing Packages from a requirements.txt File
- Clone the Repository
- Create & Activate Virtual Environment
- Install the Dependencies
- Verify Installed Packages
- Run the Project

Installing Packages from a requirements.txt File

Scenario:

Let's take this GIT Repo as an example and clone it



Now you need to **install** the **dependencies** listed in **requirements.txt**.

Link of github repo → https://github.com/gamutx/my_project

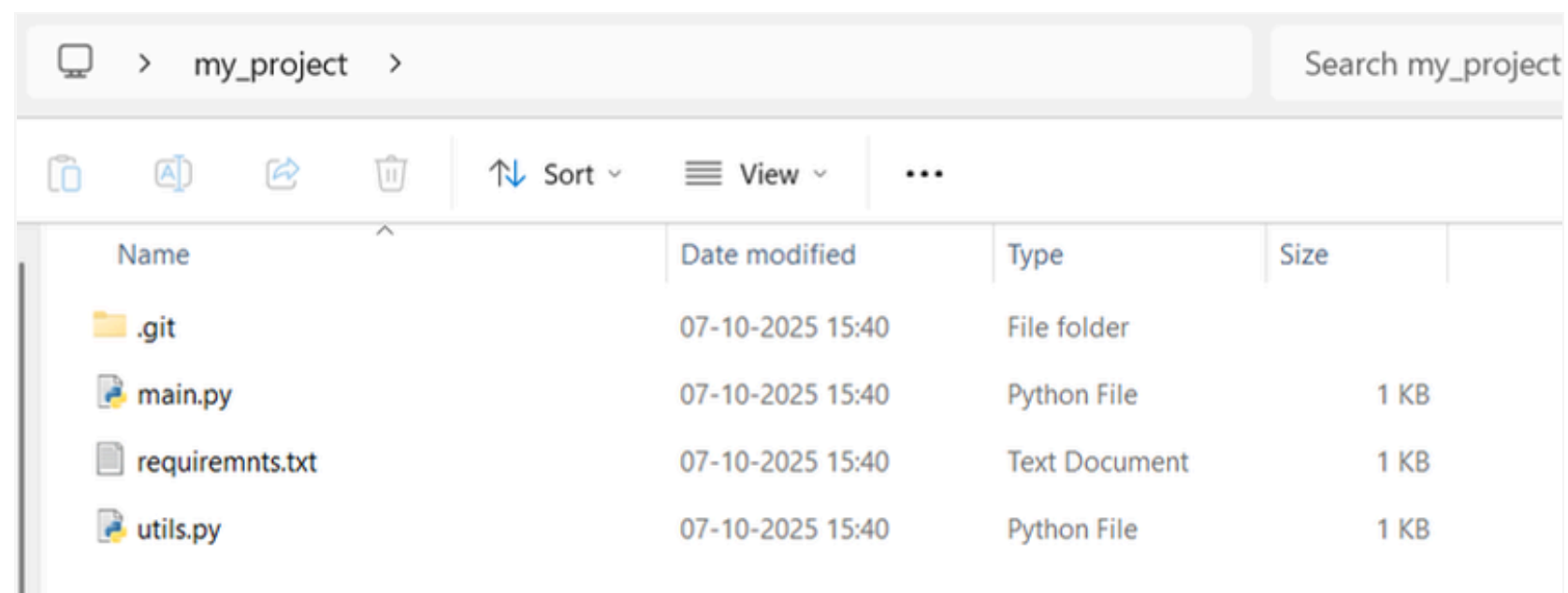
Clone the Repository

First, if you **haven't** cloned the repository yet, you can do it by using this command:

```
MINGW64:/c/Users/pralhad/Desktop
pralhad@LAPTOP-KJQAVOVJ MINGW64 ~/Desktop
$ git clone https://github.com/Pralhad-Nasane/my_project -----> clone repo
Cloning into 'my_project'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.

pralhad@LAPTOP-KJQAVOVJ MINGW64 ~/Desktop
$ |
```

Your folder should now look like this:



The screenshot shows a Windows File Explorer window with the address bar set to 'my_project'. The search bar contains 'Search my_project'. The toolbar includes icons for file operations and a 'Sort' dropdown. The main area displays a table of files and folders.

Name	Date modified	Type	Size
.git	07-10-2025 15:40	File folder	
main.py	07-10-2025 15:40	Python File	1 KB
requiremnts.txt	07-10-2025 15:40	Text Document	1 KB
utils.py	07-10-2025 15:40	Python File	1 KB

Create & Activate Virtual Environment

Create Virtual Environment

- Before installing dependencies, it's best to create a **virtual environment**.
- This keeps your project's libraries isolated from your system **Python installation**.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pralhad\Desktop\my_project> python -m venv .venv
PS C:\Users\pralhad\Desktop\my_project>
PS C:\Users\pralhad\Desktop\my_project>
PS C:\Users\pralhad\Desktop\my_project> .venv\Scripts\activate
(.venv) PS C:\Users\pralhad\Desktop\my_project>
(.venv) PS C:\Users\pralhad\Desktop\my_project>
(.venv) PS C:\Users\pralhad\Desktop\my_project>
```

creates a .venv/ folder that contains:

- A local copy of the Python interpreter
- A private copy of pip
- A location for project-specific packages

Activate the Virtual Environment

indicates that we are now inside the virtual environment.

.venv is the conventional name, but you can name it anything.

Install the Dependencies

Once your virtual environment is activated, install the required packages from the **requirements.txt** file

Content of requirement.txt

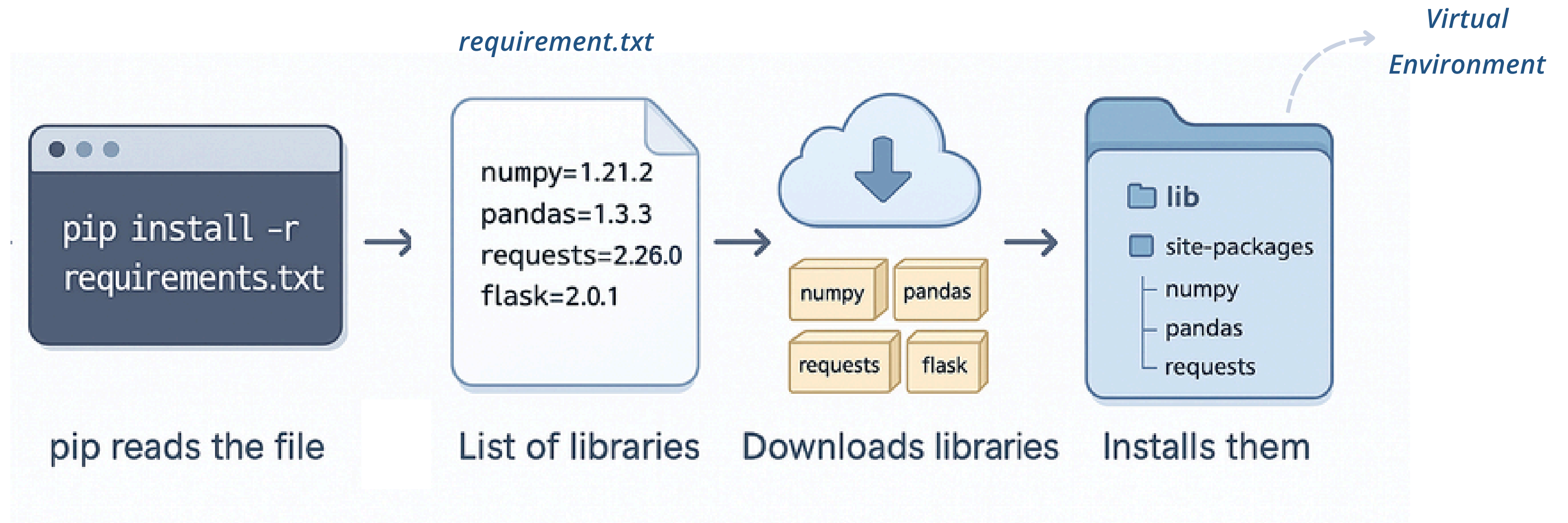
```
requirements.txt
1  numpy==1.21.2
2  pandas==1.3.3
3  requests==2.26.0
4  flask==2.0.1
```



```
(.venv) PS C:\Users\pralhad\Desktop\my_project> pip install -r requirements.txt
Collecting numpy==1.21.2
  Using cached numpy-1.21.2-cp39-cp39-win_amd64.whl (14.0 MB)
Collecting pandas==1.3.3
  Downloading pandas-1.3.3-cp39-cp39-win_amd64.whl (10.2 MB)
    |████████████████████| 10.2 MB 3.3 MB/s
Collecting requests==2.26.0
  Downloading requests-2.26.0-py2.py3-none-any.whl (62 kB)
    |████████████████████| 62 kB 4.8 MB/s
Collecting flask==2.0.1
  Downloading Flask-2.0.1-py3-none-any.whl (94 kB)
    |████████████████████| 94 kB 6.4 MB/s
Collecting python-dateutil<2.7.0,=>2.6.0
```



Behind the Scene



Verify Installed Packages

To **confirm** that the **dependencies were installed** correctly, run:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• (.venv) PS C:\Users\pralhad\Desktop\my_project> pip list
Package            Version
-----
certifi             2025.10.5
charset-normalizer  2.0.12
click               8.1.8
colorama            0.4.6
Flask                2.0.1
idna                3.10
itsdangerous        2.2.0
Jinja2              3.1.6
MarkupSafe          3.0.3
numpy               1.21.2
pandas              1.3.3
pip                 21.2.4
python-dateutil     2.9.0.post0
pytz                2025.2
requests            2.26.0
setuptools          58.1.0
six                 1.17.0
urllib3             1.26.20
Werkzeug            3.1.3
WARNING: You are using pip version 21.2.4; however, version 25
You should consider upgrading via the 'C:\Users\pralhad\Desktop
• (.venv) PS C:\Users\pralhad\Desktop\my_project>
```

(Optional) Check Manually

You can also look inside the **site-packages** folder:

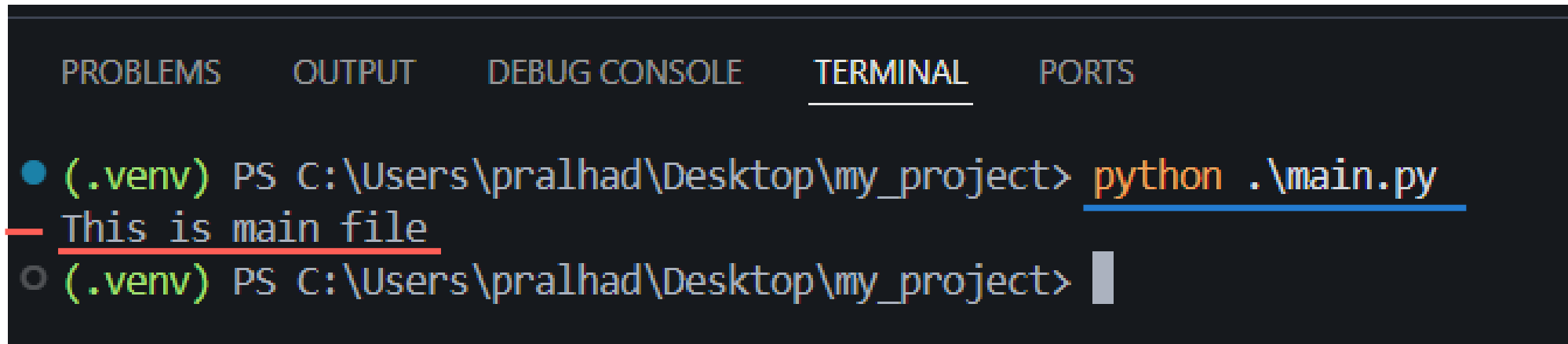
- **Windows** → `.venv\Lib\site-packages`
- **macOS/Linux** → `.venv/lib/python3.x/site-packages`

This is where **all installed libraries live**.

Indicates all packages currently installed in the **active virtual environment**

Run the Project

Once everything is set up, **run your script using:**



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal content is as follows:

```
(.venv) PS C:\Users\pralhad\Desktop\my_project> python .\main.py  
This is main file  
(.venv) PS C:\Users\pralhad\Desktop\my_project> |
```

output



Setting Up Virtual Environment and Running the Python Script



Objective

Set up a virtual environment, install required libraries, and run a Python script to make sure everything is working correctly.



Instructions:

Here's what I want you to do in this task:

- Clone the project repository to your local machine.
- Create a virtual environment inside the project folder.
- Activate the virtual environment.
- Install dependencies from the **requirements.txt** file.
- Run the main.py script to verify the setup.
- Deactivate the virtual environment.

Challenges Without Environment Management

- Challenges Without Environment Management
- How Virtual Environments Solve These Challenges

Challenges Without Environment Management

Working without virtual environments can lead to serious issues in **development**, **collaboration**, **testing**, and **deployment**.

1. Dependency Conflicts

- **Scenario:** You're working on **multiple projects** that require different versions of the **same library**.
- **Problem:** Installing one version globally can break other projects.
- **Example:**
 - Project A needs **Django 3.2**
 - Project B needs **Django 4.0**
 - → Only one can be installed **globally at a time**.

2. Packages Pollution

- **Scenario:** You **install libraries** globally for a **single project**.
- **Problem:**
 - Over time, Your **global Python environment** gets **cluttered** with packages from **multiple projects**.
 - making it difficult to manage or even know what's installed.
- **Result:** **Unused** or **conflicting packages** can **slow** down development or **cause errors**.

3. Portability Issues

- **Scenario:** You share your code with a teammate or another Developer.
- **Problem:** The other developer may not have the same libraries installed globally, or the versions may differ.
- **Example:**
 - You use **numpy 1.23**
 - They have **numpy 1.20**
 - → Leads to **errors** or **unexpected behavior**

4. Deployment Failures

- **Scenario:** You deploy **your script** or **app** to a **server or render farm**.
- **Problem:** The server may not have the required packages or versions installed.
- **Result:**
 - Application crashes or **fail** to **start**.
 - spend extra time **debugging** and **setting up the server**.

5. Testing Challenges

- **Scenario:** You write automated **tests** for **your project**.
- **Problem:** Tests may fail locally but pass in **CI/CD pipelines** (or vice versa) due to differences in installed libraries or their versions.
- **Result:** Debugging these discrepancies becomes **time-consuming** and **frustrating**.

6. Lack of Isolation

- **Scenario:** You **install** or **update** a library to try something new.
 - **Problem:** That change affects **all other projects** using the same **global library**.
 - **Result:** potentially breaking **unrelated projects**.
- 

How Virtual Environments Solve These Challenges

Isolation:

- Each project gets its **own environment** with its **own set of libraries**.

Portability :

- Dependencies are defined in a **requirements.txt** file, making it easy for others to **recreate** the environment.

Version Control :

- Specific library versions can be installed **without affecting** other **projects**.

Clean Global Environment :

- Keeps your global Python installation **free** of **unnecessary libraries**.

By using virtual environments, you can avoid these challenges and ensure a **smoother development workflow**.

Best Practices for Using Virtual Environments

- Create a Virtual Environment for Every Project
- Keep Virtual Environments Organized
- Activate and Deactivate Virtual Environments Properly
- Install Only the Necessary Packages
- Use Version Control for Your Code and Dependencies
- Keep Dependencies Up to Date

Create a Virtual Environment for Every Project

Always create a **separate** virtual environment per project.

```
python -m venv .venv
```

Why ?

- Keeps dependencies isolated
- Prevents version conflicts
- Avoids polluting the global Python environment

Tip: Every project, whether small or large, should have its own virtual environment.

Keep Virtual Environments Organized

- To maintain **clarity** and **avoid confusion**, **store** your virtual environments in a **dedicated directory**.
- For example, you might want to **store** all virtual environments in a folder like **~/environments/** or **~/venvs/**.

Why ?

- Makes it easier to manage multiple environments
- Reduces confusion across teams or machines

Naming Tip:

- Use clear names like:
 - project_name_venv
 - toolkit_env
 - maya_render_env

Activate and Deactivate Virtual Environments Properly

- Before working on your project, always activate your **virtual environment**.
- This ensures you're using the correct **Python interpreter** and **libraries** for your project.

Activate the environment:

```
source ~/environments/myprojectenv/bin/activate  
.\venv\Scripts\activate # Windows
```



Deactivate the environment:

When you're done working, run:

```
deactivate
```

This switches your terminal back to using the **global Python environment**.

Why?

- **Activation** → ensures that you're working in the **correct context**,
- **deactivating** → prevents accidental modifications to **your environment**.

Install Only the Necessary Packages

- Keep your virtual environment clean by installing **only the libraries your project actually uses**.
- Avoid adding unnecessary or unrelated packages.

Use a requirements.txt file:

Store all required packages in a **requirements.txt** file so that the environment can be set up consistently.

```
pip install -r requirements.txt
```

Why This Matters

- Reduces clutter and bloat in the environment
- Speeds up project setup and deployments
- Makes it easier to track and manage dependencies
- Prevents issues from unused or outdated packages

Use Version Control for Your Code and Dependencies

Always use **version control** (like Git) not just for your code, but also for your dependency file typically **requirements.txt**.

What to Track in Git

- Project source code (**.py**, **.pyc**, etc.)
- **requirements.txt**
- Any configuration or setup files (**.env.example**, **README.md**, etc.)

⚠ Do **not** commit the actual virtual environment folder (**.venv/**). Add it to **.gitignore**.

Why This Matters

- Ensures that everyone on the team uses the **same library versions**
- Makes setup consistent across systems and environments
- Helps you **rollback** if a package update breaks the project
- Supports automated setups (**CI/CD**, testing environments, etc.)

Keep Dependencies Up to Date

- Check for outdated packages:

```
pip list --outdated
```

- Update packages:

```
pip install --upgrade <package-name>
```

Why This Matters

- Keeps your project secure
- Avoids bugs fixed in newer versions
- Ensures compatibility with modern Python features

Caution:

- Use version pinning (**package==x.y.z**) to avoid unintended breakage during updates.

Troubleshooting for Virtual Environments

- Virtual environment not activating
- Dependencies not installing
- Conflicts with Global Dependencies
- Path Issues
- Virtual Environment Not Being Used
- Outdated packages
- Conflicts between different environments
- Why is a Virtual Environment Required?

Virtual environment not activating

Scenario: You run the activate command, but the environment doesn't activate. No **(.venv)** appears in the terminal.

Issue:

- **Incorrect command** for the operating system
- Wrong folder path
- Terminal **doesn't recognize** the environment
- **Shell** alias or execution permission issues

Solution:

- Use the **correct activation command**:
- **Windows** → `.\venv\Scripts\activate`
- **macOS/Linux** → `source venv/bin/activate`
- Make sure you're inside the project folder where **.venv/** was created
- Restart the terminal
- Ensure the **activate** script exists and is executable

Virtual environments don't activate automatically — they must be **manually activated each session**.

Dependencies not installing

Scenario:

You run **pip install** or **pip install -r requirements.txt**, and installation fails or throws errors.

Issue:

- Misspelled package name
- Network issues
- Outdated version of pip
- Conflicting versions in **requirements.txt**

Solution:

- First, **activate** the **virtual environment**
- Check your internet connection
- Upgrade pip → `pip install --upgrade pip`
- Double-check package names and syntax
- If using **requirements.txt**, look for incompatible version constraints

Use **pip list** after install to **confirm what was added**.

Conflicts with Global Dependencies

Scenario:

Your global Python installation may **have dependencies** that **conflict** with those in your **virtual environment**.

Issue:

You observe **unusual behavior** or **errors** due to dependency conflicts.

Solution:

- Confirm you're inside the **correct virtual environment**
- Isolate your virtual environment by not mixing packages **installed globally** and those installed in the **virtual environment**.
- If necessary, delete and recreate the virtual environment:

Always activate your environment **before installing** or **running anything**.

Path Issues

Scenario: Scripts can't find modules, or you get **errors** like **ModuleNotFoundError**.

Issue:

- You're not in the **right directory**
- Virtual environment is **not active**
- Python path or shebang is pointing to the **wrong interpreter**

Solution:

- Confirm you're in the project root (check with **pwd** or **cd**)
- Activate the environment
- Ensure scripts are using the correct shebang:

```
#!/usr/bin/env python
```

In some **IDEs** or **shell setups**, Python may default to the global environment. Always verify the **active interpreter**.

Virtual Environment Not Being Used

Scenario:

Sometimes, you might run code without realizing you're not using the **virtual environment**.

Issue:

- You forgot to activate the **virtual environment**
- **IDE** or **terminal** is using **global Python**

Solution:

- Check the terminal prompt — it should show **(.venv)**
- If using VS Code:
 - Click the Python interpreter in the bottom corner
 - Select the one from **.venv**

Running **python** without activation uses system Python by default — even inside your project folder.

Outdated packages

Scenario: code throws **errors**, behaves **unpredictably**, or **lacks** newer features — even though it should be working.

Issue:

- One or more libraries are **outdated**
- You're missing bug fixes, performance improvements, or compatibility updates

Solution:

- Check for outdated packages:

```
pip list --outdated
```

- Upgrade specific packages:

```
pip install --upgrade <package-name>
```

- (Optional) Upgrade all packages carefully — one by one, especially in production tools.

Key Notes:

Keep your **requirements.txt** updated after making changes: `pip freeze > requirements.txt`

Conflicts between different environments

Scenario: Changes in one project affect another — e.g., after installing a package for Project A, Project B starts failing.

Issue:

- **Shared** or **overlapping** virtual environments
- Using global Python for **multiple projects**
- Incorrect interpreter selected in the **terminal** or **IDE**

Solution:

- **Use a separate virtual environment for each project.** Never share .venv folders.
- **Activate the correct environment** before installing or running anything.
- **Verify the Python interpreter path:**

```
which python
where python
```

It should point to the correct **.venv** for the current project.

One project, one environment — this ensures full isolation and avoids dependency conflicts.

Why is a Virtual Environment Required?

1. Dependency Isolation

The Problem

- Each project may rely on **different versions of the same library**.
- **Example:**
 - Project A requires **Django 3.2**
 - Project B requires **Django 4.0**

If installed globally, **one version overwrites the other**.

This breaks whichever project no longer has its **required version**.

✓ Solution:

- A **virtual environment** keeps dependencies **project-specific**, so:
 - Each project installs its own version of Django
 - No overlap or conflict between tools

2. Reproducibility

The Problem

- Your script works on your machine, but not on your colleague's.
- **Example:**
 - You use the **colorama** library to format console output
 - Your colleague runs the script and sees:

```
ModuleNotFoundError: No module named 'colorama'
```

This happens because they **don't have the dependency installed**.

✓ Solution:

- With a **virtual environment**:
 - All required libraries are tracked in one place
 - You can share them via a **requirements.txt** file:
- Others can recreate your environment exactly:

```
pip freeze > requirements.txt
```

```
pip install -r requirements.txt
```

This is essential for **team collaboration**, **render farm deployment**, or **publishing internal tools**.

3. Clean Development Environment

The Problem

- Installing libraries **globally** can:
 - Pollute your system with **unused packages**
 - Cause **hidden version conflicts** between projects
- This makes debugging harder and the system **harder to maintain**.

✓ The Solution:

- Virtual environments:
 - Keep libraries **isolated per project**
 - Avoid polluting the system Python installation
 - Make your setup easier to **audit** and **manage**

4. Simplifying Deployment

The Problem

- Production systems (like render farms or servers) may not have the same setup as your machine.
- **You risk:**
 - Missing packages
 - Incompatible versions
 - Runtime errors that didn't occur during development

✓ The Solution:

- A **virtual environment** allows you to:
 - **Mirror development and production setups**
 - Ensure all required packages are present
 - Deploy code with confidence that it will behave the same

Especially useful when integrating with render farm job scripts or submitting tools to shared **DCC environments** like **Maya**, **Houdini**, or **Nuke**.

5. Flexibility in Python Versions

The Problem

- Different projects may require different Python versions.
- **Example:**
 - **Tool A** needs **Python 3.8** (because Maya 2022 uses it)
 - **Tool B** needs **Python 3.10** (for new syntax features)
- Installing and switching global Python versions:
 - Is time-consuming
 - Can break system tools or other DCCs

✓ The Solution:

- You can create a virtual environment using a **specific Python version:**

```
python3.8 -m venv venv_a  
python3.10 -m venv venv_b
```

- **Each project:**
 - Runs with its required Python version
 - Doesn't affect or rely on system-wide Python

6. Preventing System-wide Changes

The Problem

- Installing packages globally may:
 - Affect system tools that rely on a **specific library**
 - Cause breakages in Python-based **DCCs** or **utilities**
- This is risky, especially on **shared workstations** or **production machines**.

✓ The Solution:

- A virtual environment:
 - Keeps all changes **local to the project**
 - Avoids unintended consequences to your system

This is a best practice for **tool reliability** and **pipeline stability**.

Virtual environment is required to:

- **Isolate project dependencies** and avoid conflicts.
 - **Ensure reproducibility** for shared or deployed projects.
 - Maintain a **clean and organized development setup**.
 - Use **different Python versions for different projects**.
 - **Avoid system-wide changes** that can disrupt other applications.
- 

General Queries

Difference Between ENV Path and Virtual ENV

Virtual Environment :

A project-specific **Python workspace** that keeps code and **dependencies isolated**.

What It Includes:

- Its own **Python interpreter**
- Its own **installed libraries** (e.g., **numpy**, **requests**)
- Optional scripts or binaries
- Typically located in your project folder (e.g., **.venv/**)

Example:

- **Project A:** uses **numpy==1.24**
- **Project B:** uses **numpy==1.21**
- → Each project has its own environment, so they don't interfere.

Environment PATH (System PATH Variable)

A **system-wide OS** setting that tells your terminal **where to look for executables**.

What It Affects:

- Lets you run commands like **python, pip, git**, etc., **from any folder**
- **Without** it, you'd need to **provide full paths to every tool**

Without PATH:

```
c:\Python311\python.exe script.py
```

OR

Without PATH:

```
python script.py
```

Purpose:

- Makes system-wide commands **easier to run**
- Used to locate **global tools**, not manage **per-project setups**

Difference Between Activating and Running a Virtual ENV

These **two** concepts are related — but not the same. Let's clarify what each one means and when it matters.

◆ Seeing (.venv) in the Terminal

- If you see something like:

```
(.venv) C:\Users\you\project-folder>
```

- It means:
 - The virtual environment is **activated**
 - **python** and **pip** now point to the environment inside **.venv/**, not the system Python
- But there's more to the story.

◆ Activating the Virtual Environment

Activation means you're telling the terminal to **use the virtual environment's Python and packages**.

- Windows → `.\.venv\Scripts\activate`
- macOS/Linux → `source .venv/bin/activate`

◆ Running Python Without Activation

We can still use **the virtual environment** **without** activating it **manually**, especially inside an **IDE** like **VS Code**.

Example:

- You select the **.venv** interpreter inside VS Code
 - You run **main.py** — and it uses **.venv/python**
 - But the **terminal is still using global Python**
- 

Q - What Happens If We Don't Activate the Virtual Environment Before Installing Packages?

- When you run:

```
pip install some-library
```

- **without activating** the virtual environment, pip doesn't know about your **.venv**.
- It installs the package into the **global Python environment**, not your project.

Why That's a Problem

Issue

- **Package installed globally**
- **ModuleNotFoundError**
- **Global environment affected**

Impact

- Not available inside your virtual environment
- Your script fails when run from **.venv**
- Other projects or system tools may break or conflict

Thank You

